PAYLOAD-AWARE TWO-TIER UAV ROUTE PLANNING FOR LAST-MILE DELIVERY

PHASE I REPORT

Submitted By

THIRUMURUGAN RA 3122235001149

MEGHNA MANIMARAN 3122235001079

SHRIKA SP THOTA 3122235001125

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



Department of Computer Science and Engineering

Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University)
Kalavakkam - 603110

October 2026

Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University)

BONAFIDE CERTIFICATE

Certified that this project report titled **"PAYLOAD-AWARE TWO-TIER UAV ROUTE PLANNING FOR LAST-MILE DELIVERY"** is the *bonafide* work of **THIRUMURUGAN RA (3122235001149)**, **MEGHNA MANIMARAN (3122235001079)** and **SHRIKA SP THOTA (3122235001125)** who carried out the project work under my supervision.
Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Dr. T.T. MIRNALINEE
HEAD OF THE DEPARTMENT
Professor
Department of CSE
SSN College of Engineering
Kalavakkam - 603 110

Dr. N. SUJAUDEEN
SUPERVISOR
Assistant Professor
Department of CSE
SSN College of Engineering
Kalavakkam - 603 110

Submitted for project viva-voce examination held on. . . . . .

EXTERNAL EXAMINER **INTERNAL EXAMINER**

ACKNOWLEDGEMENTS

We thank God almighty for giving us the strength and the patience to carry this work through.

We are deeply grateful to our guide, **Dr. N. SUJAUDEEN**, Assistant Professor, Department of Computer Science and Engineering, for his advice, his patience with our many drafts and his steady push to back every claim with a number. This work took its shape in our discussions with him.

Our sincere thanks to **Dr. T.T. MIRNALINEE**, Professor and Head of the Department of Computer Science and Engineering, for her encouragement and support. We thank **Dr. B. BHARATHI**, Project Coordinator, and the members of our review panel, whose questions at each review made the system better than it would otherwise have been.

We also thank **Dr. S. RADHA**, Principal, for the facilities and support she has provided during our course of study.

We thank the teaching and non-teaching staff of our department who helped us, directly or indirectly, during the project. Finally, we thank our parents and friends for their patience and support throughout.

THIRUMURUGAN RA MEGHNA MANIMARAN SHRIKA SP THOTA

ABSTRACT

In UAV systems for medical and emergency last-mile delivery, conflicting objectives must be balanced: higher flight risk is carried by shorter routes, more battery is drained by low-risk corridors, and the fastest paths are rarely achieved by energy-efficient routing. No single fixed trade-off serves all payload types, yet the same objective weights are applied by existing planners to every mission regardless of the payload. In existing multi-objective planners, including entropy-based, diversity-driven, and crowding-distance Pareto selection methods, objective weights are encoded as a fixed scalar before path search begins. No influence is exerted by payload properties such as fragility class, payload weight class, and delivery urgency over which Pareto-optimal route is chosen. When payloads change, weights must be manually re-tuned and planners re-run by operators, which is unworkable under field conditions. The proposed work is a Payload-Aware Two-Tier UAV Route Planning system, wherein Pareto front construction is separated from route selection and an explicit algebraic bridge is added between Tier 1 routing objectives and Tier 2 payload metadata. In Tier 1, the NAMOA* multi-objective search is executed over a three-dimensional airspace lattice built from terrain elevation, OpenStreetMap buildings and restricted zones, with wind-dependent, direction-aware edge costs, producing a Pareto-optimal route set across distance, energy consumption, and flight risk. In Tier 2, four operator-assigned payload tags are read and objective weights derived: w_{risk} is scaled by fragility class, w_{distance} by delivery urgency, and w_{energy} by payload weight class. Every Pareto point is scored by these weights via a

utility function, and the argmax is selected as the mission route. The principal contribution is a formula-driven mapping from metadata to routing weights, linking mission context directly to Pareto selection without expert intervention. Route choice is set by payload properties rather than context-blind diversity criteria. The Pareto front is stored and rebuilt only when the graph changes, its validity window expires or the weather drifts, serving all missions between updates.

Keywords: UAV route planning; multi-objective optimisation; NAMOA*; Pareto front; payload-aware selection; wind-aware risk; last-mile delivery

TABLE OF CONTENTS

ABSTRACT **iii**

LIST OF TABLES **x**

LIST OF FIGURES **xi**

LIST OF ABBREVIATIONS **xii**

1 INTRODUCTION **1**

1.1 MOTIVATION 2

1.2 BACKGROUND 4

1.2.1 Building a front versus choosing from it 4

1.2.2 Evidence from real deployments 4

1.2.3 Regulation 5

1.3 PROBLEM DEFINITION 5

1.3.1 Formal statement 6

1.3.2 Weather dependence 7

1.3.3 Inputs, outputs and success criteria 7

1.3.4 Scope 7

1.4 ORGANIZATION OF THE REPORT 8

2 LITERATURE SURVEY **9**

2.1 PAPERS RELATED TO THE PROBLEM IDENTIFIED 9

2.1.1 Pareto-optimal particle swarm optimisation with entropy selection . . . 9
2.1.2 Crowding-distance NSGA-II for urban UAV path planning 10
2.1.3 Multi-strategy improved NSGA-II for 3-D trajectory planning . . . 11
2.1.4 NSGA-II and MOPSO . . . 12
2.1.5 Exact multi-objective A* search . . . 12
2.1.6 Payload-mass-aware trajectory planning . . . 13
2.1.7 Quality indicators for Pareto fronts . . . 13
2.1.8 Effect of drone transport on medical payloads . . . 14
2.1.9 Summary of the reviewed work . . . 14
2.2 GAPS IDENTIFIED . . . 16
2.3 RESEARCH OBJECTIVES . . . 17

3 PROPOSED METHODOLOGY 19
3.1 ARCHITECTURE DESIGN . . . 19
3.2 AIRSPACE GRAPH BUILDER (MODULE 1) . . . 21
3.2.1 Environment data acquisition . . . 22
3.2.2 Lattice generation and node selection . . . 22
3.2.3 Edge construction and pruning . . . 23
3.2.4 Static risk terms . . . 24
3.2.5 Wind-aware energy model . . . 26
3.2.6 Weather-dependent risk and the go/no-go gate . . . 27
3.3 TIER 1 PARETO FRONT ENGINE (MODULE 2) . . . 27
3.3.1 Labels and dominance . . . 28
3.3.2 Admissible vector heuristic and pruning . . . 28

3.3.3 Truncation and front thinning 29
3.3.4 Why an exact method 31
3.3.5 Pareto front store and refresh 31
3.4 BASELINE PLANNERS (MODULE 3) 32
3.5 TIER 2 PAYLOAD METADATA INTERPRETER (MODULE 4) . 32
3.5.1 Semantic mapping 33
3.5.2 Weight derivation 34
3.5.3 Utility scoring and selection 35
3.5.4 Reference presets 35
3.5.5 Battery feasibility 36
3.5.6 Weather hold and decision record 36
3.6 EVALUATION AND BENCHMARKING (MODULE 5) 37
3.6.1 Front quality measures 37
3.6.2 Tier 2 evidence 38
3.6.3 Tag sweep and monotonicity 38
3.6.4 The evaluation cycle 39
3.7 MISSION EXPORT AND DASHBOARD (MODULE 6) 39
3.7.1 Mission file generation 39
3.7.2 Operator dashboard 40
3.8 SIMULATION AND SITL VALIDATION (MODULE 7) 40
3.8.1 Bundle, clearance and frame contract 40

4 EXPERIMENTAL RESULTS AND PERFORMANCE ANALYSIS 42
4.1 DATASET DESCRIPTION 42
4.1.1 Synthetic airspace graphs 43
4.1.2 Real-region lattice 44

4.1.3 Payload tag space and simulation bundle 45
4.2 ECOSYSTEM 45
4.2.1 Hardware configuration 45
4.2.2 Software environment 45
4.3 EXPERIMENTS CONDUCTED 46
4.4 PERFORMANCE ANALYSIS 47
4.4.1 Tier 1 front quality against the baselines 47
4.4.2 Effect of wind speed and direction 49
4.4.3 Directional edge costs 51
4.4.4 Contribution of each risk layer 52
4.4.5 Tier 2 selection 52
4.4.6 Simulation results 54
4.4.7 Comparison with existing work 55

5 SOCIAL IMPACT AND SUSTAINABILITY 57
5.1 SOCIAL IMPACT 57
5.2 SUSTAINABILITY 58

6 CONCLUSION AND FUTURE WORK 60
6.1 CONCLUSION 60
6.2 FUTURE WORK 61

REFERENCES 62

A CONFIGURATION PARAMETERS 65

B AUTOMATED TEST SUITES 67

C CODE ORGANISATION **69**

LIST OF TABLES

1.1 Inputs, outputs, constraints and success criteria 8

2.1 Summary of the reviewed work 15
2.2 Gaps identified and the research objectives that address them . . . 18

3.1 Risk components (every $\rho_k \in [0, 1]$) and their weights 25
3.2 Semantic mapping from payload tags to Tier 1 objectives 33
3.3 Reference payload presets and their derived weights 36

4.1 Graphs and scenarios used in the experiments 43
4.2 Environment data sources for the real-region lattice 44
4.3 Experiments carried out in Phase I so far 47
4.4 First-pass benchmark on the synthetic scenarios (HV higher is better; IGD and Δ lower are better) 48
4.5 Effect of wind direction on the S2 front (25 km/h wind, 35 km/h gust) 50
4.6 Directional edge costs on a flat lattice, 25 km/h wind from 90° . . 51
4.7 Ablation on the 6 km lattice (fixture layers, 30 km/h wind from 225°) 52
4.8 Tier 2 picks on the S2 (16 routes) and S3 (115 routes) fronts . . . 52
4.9 Comparison with the reviewed planners 56

A.1 Main airspace graph and cost model parameters 65
A.2 Front store and Tier 2 parameters 66

B.1 Test suites and what they verify 67

C.1 Source files and the modules they implement 69

LIST OF FIGURES

3.1 Methodology overview: Tier 1 Pareto front construction feeding Tier 2 payload-aware route selection, with mission output, UAV control and evaluation 20
3.2 System architecture. Tier 1 builds and stores the front per region and weather window; Tier 2 selects a route per payload tag; Modules 6 and 7 export and fly it 21
3.3 Gazebo and ArduPilot SITL simulation pipeline 41

4.1 Hypervolume per method and scenario (first pass) 49
4.2 Front size, minimum energy and minimum risk on S2 as a 25 km/h wind turns through 360°; the dashed line is calm air 51
4.3 Tier 2 routes for the four presets on the 2 km lattice 54
4.4 Gazebo Harmonic and ArduPilot SITL flying an uploaded Tier 2 mission in AUTO: MAVProxy console (left), QGroundControl mission view (right) 55

LIST OF ABBREVIATIONS

AGL	Above Ground Level
AMSL	Above Mean Sea Level
DEM	Digital Elevation Model
GPS	Global Positioning System
HV	Hypervolume
IGD	Inverted Generational Distance
MAVLink	Micro Air Vehicle Link
MOPSO	Multi-Objective Particle Swarm Optimisation
NAMOA*	New Approach to Multi-Objective A*
NFZ	No-Fly Zone
NSGA-II	Non-dominated Sorting Genetic Algorithm II
OSM	OpenStreetMap
POPSO	Pareto-Optimal Particle Swarm Optimisation
QGC	QGroundControl
RMS	Root Mean Square
SDF	Simulation Description Format
SITL	Software In The Loop
SORA	Specific Operations Risk Assessment
UAV	Unmanned Aerial Vehicle
WMO	World Meteorological Organization

CHAPTER 1

INTRODUCTION

Unmanned aerial vehicles (UAVs) are a practical option for last-mile delivery where ground transport is slow or impossible: medical supplies for remote clinics, emergency drug drops after a disaster, and diagnostic samples that should not spend an hour shaking in the back of a van. In each case the UAV flies from a dispatch hub to a delivery point through airspace that has buildings, terrain, restricted zones and wind in it. A good route has to balance three things that pull against each other: how far the drone flies, how much battery it uses, and how much of the flight is spent in risky air.

Multi-objective path planning was built for this kind of conflict. A multi-objective planner keeps distance, energy and risk separate during the search and returns a Pareto-optimal front, which is a set of routes where none can be improved on one objective without getting worse on another [7]. The front tells the operator which trade-offs exist. It does not say which route to fly.

The usual shortcut is to fold the three objectives into one weighted sum and run an ordinary planner such as A* on the resulting scalar cost. The operator picks the weights before each flight to express what the mission cares about. For one fixed mission type this works. It stops working once the payload changes, because emergency medicine needs a very different weight profile from a fragile blood sample or a box of relief supplies [4]. Re-tuning the weights and running the search again between missions is slow, needs someone who understands the planner, and is hard to justify in the middle of an emergency.

This project proposes a Payload-Aware Two-Tier UAV Route Planning system that removes that step. Tier 1 builds the Pareto front for a delivery region once, with an exact multi-objective graph search over a three-dimensional airspace lattice whose edge costs depend on terrain, buildings, restricted zones and the direction of the wind. The front is stored and reused until the graph changes, the front expires or the weather drifts. Tier 2 runs per mission. It reads four payload tags entered by the operator, turns them into objective weights with a fixed formula, scores every route on the stored front and returns the best one in well under a millisecond. The chosen route is exported as an ArduPilot mission and flown in a Gazebo and ArduPilot software-in-the-loop (SITL) simulation.

This report covers Phase I: the software system, its benchmarks against single-objective A* and MOPSO, and the first simulated flights. Phase II moves the same software onto a physical quadcopter.

1.1 MOTIVATION

Three delivery scenarios that come up again and again in medical and emergency drone work motivate this project, and their routing needs do not agree with each other.

Fragile medical samples such as tissue biopsies and blood specimens must arrive intact, and an operator will want them kept out of rough air when a smoother corridor exists. The priority is low flight risk: little time in turbulent or crosswind-exposed corridors and few sudden altitude changes. A somewhat longer but much calmer route is better.

Emergency medicine such as insulin or antivenom has a strict time window. The priority is the shortest route, so that transit time is as low as possible, and risk and energy come second. If the delivery window is fifteen minutes, the shortest path wins.

Disaster-relief supplies such as water purification tablets and ration packs are heavier than medical payloads, and they are neither fragile nor urgent. Here the priority is low battery drain, so that one charge covers as many drops as possible.

A real delivery service sees all three, sometimes within the same hour. A planner with fixed weights treats them identically, and one that lets the operator change the weights pushes an optimisation task onto someone dispatching a drone in a hurry. We could not find a UAV delivery planner that switches between these routing strategies from a description of the payload, without an expert in the loop and without running the search again. That is what motivated the two-tier design.

A second motivation is wind. None of the planners we reviewed resolve wind direction per edge, yet for a quadcopter cruising at 36 km/h a 25 km/h wind is not a detail: flying into it costs far more energy than flying with it, and crosswind makes the ride rough. Section 4.4.2 shows that at a fixed wind speed, the direction alone changes the energy of the cheapest available route by 27.5%. A planner blind to direction gets this wrong on any windy day, and so does any payload-aware choice built on top of it.

1.2 BACKGROUND

1.2.1 Building a front versus choosing from it

Pareto fronts for routing are built either by exact graph search, such as NAMOA* [11], which keeps every non-dominated partial path at each node, or by population methods such as NSGA-II [6] and MOPSO [5], which are faster on large spaces but stochastic. Building the front is well studied. Picking one route from it gets far less attention: the usual options are a crowding-distance rule [6, 18], an entropy criterion followed by a hand-set weighted sum [4], or a person. None of them looks at what the drone is carrying. Payload does appear in the literature as continuous mass inside the energy model [21], which changes the costs, not the choice between trade-offs. What is missing is any use of payload metadata to select a point on a precomputed front.

1.2.2 Evidence from real deployments

Real deployments show where payload matters, and also how thin the public evidence still is. Swiss Post flew laboratory samples between hospitals on Matternet quadrotors until two incidents in 2019 led it to suspend the service. In January a short circuit cut power to the GPS and the drone made an emergency landing on Lake Zürich. In May a parachute tether snagged on the airframe and the 22-pound drone fell into woods near a group of kindergarten children [2]. Both were equipment failures, so they say nothing about route choice. They do show that laboratory samples are routinely flown over water and populated ground, where the choice of corridor has consequences.

Laboratory studies of medicines and blood samples flown by drone found no damage on ordinary flights [14, 19, 20], but none of them varied the route (Section 2.1), so in this work the fragility tag is a precaution the operator sets, not a measured degradation effect.

Zipline's blood-delivery network in Rwanda serves hospitals up to about 80 km from its distribution centre, and each aircraft follows a predetermined flight plan [9]. Its engineers treat payload weight as a limit on range. That is the energy side of the problem Tier 2 addresses: a heavier load shortens how far the drone can go, so the best route depends on what is on board.

1.2.3 Regulation

In India, small civil drones fly under the Drone Rules, 2021 [13], which treat 120 m above ground as the ceiling for flight without special permission. Ground risk is commonly ranked with the SORA framework [10]. Our airspace model uses both.

1.3 PROBLEM DEFINITION

Existing UAV route planners either apply a fixed geometric selection rule over the Pareto front that ignores the payload, or fold the objectives into one hand-tuned weighted sum before the search begins, which fixes the trade-off regardless of what is being carried. When the payload changes, an expert has to re-tune the weights and run the search again, because no explicit mapping exists from a payload description to the trade-off it implies. In the field, that expert is not available.

1.3.1 Formal statement

We are given:

1. an airspace graph $G = (V, E)$ in which every edge carries a three-objective cost vector $\mathbf{c}(e) = [d(e), \varepsilon(e), \rho(e)]$ for flight distance, estimated energy and flight risk;

2. a Pareto front $\mathcal{P}$ of non-dominated routes computed in advance over G; and

3. a payload metadata tuple $(F, W, U, P_{\text{lvl}})$ holding fragility class $F \in \{1, \ldots, 5\}$, payload weight class $W \in \{1, 2, 3\}$, delivery urgency $U \in \{1, 2, 3\}$ and priority level $P_{\text{lvl}} \in \{1, \ldots, 5\}$.

The problem is to select the route $p^* \in \mathcal{P}$ that best meets the requirements implied by the payload metadata, without expert re-tuning and without running the multi-objective search again.

The inputs are values an operator can assign from domain knowledge ("this vial is fragile, $F = 5$; the delivery is not urgent, $U = 1$"). Selection must be deterministic and reproducible, and it must finish within one second of the tags being entered. Three further conditions follow from the gaps identified in Chapter 2. The selected route must depend on $(F, W, U, P_{\text{lvl}})$ and not on the geometry of the front alone. The front must be reused across missions between refreshes instead of being rebuilt each time. The map from $(F, W, U, P_{\text{lvl}})$ to p^* must be an explicit formula, not a set of rules written by an expert and not a lookup table.

1.3.2 Weather dependence

The edge costs depend on the weather. Writing ω for a weather snapshot (wind speed, wind direction, gust, rain and visibility), the cost of an edge is $\mathbf{c}(e;\omega) = [d(e), \varepsilon(e;\omega), \rho(e;\omega)]$ and the front becomes $\mathcal{P}(\omega)$. Distance stays the same, but energy and risk change with how the wind meets each edge. The selection problem itself does not change. What changes is that a stored front is only valid while ω stays close to the snapshot it was built for. That is why the front is refreshed on a schedule and on weather drift. When ω crosses the go/no-go limits, no route is selected at all and the mission is held.

1.3.3 Inputs, outputs and success criteria

Table 1.1 lists what the system takes in, what it returns and how each part is judged.

1.3.4 Scope

Phase I, covered by this report, delivers the full software system and its simulation. Phase II moves the system onto a Pixhawk-equipped F450 quadcopter with a Raspberry Pi 5 companion computer and validates it with outdoor flights. The Tier 1 front is treated as valid for a bounded window. It is rebuilt when the window expires or when the weather drifts past set thresholds, and every mission inside the window is served by Tier 2 alone. The project does not cover multi-UAV coordination, avoidance of moving obstacles during flight, or real-time replanning.

TABLE 1.1: Inputs, outputs, constraints and success criteria

Item	Definition
Region input	Centre point and box size; terrain (DEM), OpenStreetMap buildings, restricted zones, water and land use for that box
Weather input	Snapshot ω from Open-Meteo: wind at 80 m, gust, rain, visibility, WMO weather code
Mission input	Depot, destination and payload tuple $(F, W, U, P_{\text{lvl}})$
Tier 1 output	Stored Pareto front $\mathcal{P}(\omega)$ with graph hash, weather snapshot and expiry time
Tier 2 output	Selected route p^* with waypoints (latitude, longitude, altitude), weights, ranks and point type, or a HOLD advisory
Mission output	QGC WPL 110 `.waypoints` and QGroundControl `.plan` files
Hard constraints	Cruise at least 60 m above ground and 15 m above roofs; at most 120 m above ground; no entry into restricted zones; route energy within 80% of battery capacity
Tier 1 criteria	Hypervolume and IGD at least as good as every baseline on each scenario; no dominated or duplicate routes in the front
Tier 2 criteria	Selection under 1 s; zero monotonicity violations over all 225 tag combinations; non-extreme picks for mixed payloads
Simulation criteria	Cross-track error, clearance margin and landing offset of the flown route in SITL and Gazebo

1.4 ORGANIZATION OF THE REPORT

The report has six chapters, followed by the references and three appendices. Chapter 1 sets out the motivation, background and problem. Chapter 2 reviews the closest papers, the gaps in them and our research objectives. Chapter 3 describes the architecture and its seven modules with the algorithms behind them. Chapter 4 covers the datasets, the hardware and software, the experiments and what the results show. Chapter 5 discusses the social impact and sustainability of the work. Chapter 6 concludes and lists the work planned for the rest of Phase I and for Phase II. The references follow, and the appendices give the main configuration parameters, the automated test suites and the code organisation.

CHAPTER 2

LITERATURE SURVEY

Multi-objective UAV path planning draws on both graph search and evolutionary optimisation [7]. We read the literature with one question in mind: once a planner has a set of good routes, how does it pick the one to fly, and does the payload have any say in that choice? This chapter reviews the papers closest to that question, the algorithms our system builds on or competes with, and the small body of work on how drone transport affects medical payloads. It then states the gaps we found and the objectives that follow from them.

2.1 PAPERS RELATED TO THE PROBLEM IDENTIFIED

2.1.1 Pareto-optimal particle swarm optimisation with entropy selection

Chokkalingam and Ganesan [4] propose a Pareto-optimal particle swarm optimisation (POPSO) algorithm for UAV path planning in changing environments. Their framework combines four ideas. The swarm is initialised with a mix of grid-based and random particles to start from a spread of paths. The cost function covers path length, energy, an inverse safety term, a moving-obstacle penalty and a weather factor. The inertia weight and learning

coefficients of the swarm are updated during the run, so the search explores more when it stalls and exploits more when it is improving. Finally, a hybrid Pareto-entropy mechanism ranks archive members by Pareto rank and by an entropy measure computed from distances between solutions, which keeps the archive diverse. On real terrain images with added obstacles it produced well-distributed fronts and better path efficiency and safety than the methods it was compared with.

Two details matter for our work. The weather factor is $W = \gamma(1 + v_{\text{wind}}/v_{\max})$, which depends on wind speed only, so a path flown into the wind and the same path flown with it are costed alike. And the final route is not chosen by the entropy mechanism. The paper defines an auxiliary weighted sum over the normalised objectives, with weights "typically determined empirically based on mission priorities", and uses it both for particle updates and to select the representative solution that is actually deployed. So the last step is still a hand-set weighted sum, and the payload does not enter it.

2.1.2 Crowding-distance NSGA-II for urban UAV path planning

Ren et al. [18] plan UAV paths in an urban environment with two objectives, distance and safety. The city is stored as an octree, which subdivides 3-D space hierarchically and keeps memory low in large open regions, and a safety index map scores how close each cell is to obstacles. The population is seeded with A* paths and evolved with NSGA-II. The authors point out a case where the standard crowding distance prefers a counter-intuitive point, and they redefine crowding as

the average distance to the k nearest solutions. The resulting CDNSGA-II finds well-spread, collision-free Pareto paths efficiently.

The authors argue that a weighted sum needs its weights fixed in advance, which is hard, so they build a front first and pick from it. The pick, though, is made by crowding distance, which measures how isolated a point is on the front and has no notion of urgency, fragility or payload weight.

2.1.3 Multi-strategy improved NSGA-II for 3-D trajectory planning

Deng et al. [7] address two known weaknesses of evolutionary planners in 3-D: getting stuck in local optima, and the tension between converging quickly and keeping the population diverse. Their MS-INSGA2 framework seeds the population with linearly interpolated trajectories that pass a safety check, couples a dynamic guidance step with crossover and mutation rates that adapt to the state of the population, and adds a local search that refines the Pareto front and smooths the trajectories. Three objectives are optimised: path length, collision risk from terrain and cylindrical obstacles, and smoothness measured as the change in curvature between segments. On the ZDT, DTLZ and WFG benchmark suites, with a Wilcoxon signed-rank test, the framework outperformed its baselines, and on two 3-D planning tasks it beat eleven other algorithms, lowering the average fitness by 27.87% and 32.06% and the best fitness by 6.91% and 14.18%.

The paper is about search quality and trajectory smoothness. Payload and delivery context do not appear among the objectives, and wind is listed as future work rather

than modelled. As with the other evolutionary planners, the front is rebuilt for each planning task.

2.1.4 NSGA-II and MOPSO

NSGA-II [6] is still the reference evolutionary algorithm for several objectives. It brought fast non-dominated sorting, elitism by merging parents and offspring, and the crowding distance, which prefers solutions in sparse regions of the front. MOPSO [5] extended particle swarm optimisation with an external archive of non-dominated solutions, leader selection on an adaptive grid and a mutation operator. Both are fast, stochastic and widely used as baselines. Our MOPSO baseline adapts the second to graph routes (Section 3.4), and the crowding distance is one of the competing selection rules that Tier 2 is compared against (Section 3.6).

2.1.5 Exact multi-objective A* search

Mandow and Pérez de la Cruz [11] analyse NAMOA*, a multi-objective extension of A*. Instead of one cost per node, NAMOA* keeps a set of non-dominated cost vectors (labels) per node and expands labels, not nodes. With an admissible heuristic it returns the complete set of Pareto-optimal solution costs. The paper proves that, when the heuristic is consistent, NAMOA* expands no more labels than any other admissible multi-objective search algorithm given the same heuristic, and that a better-informed heuristic never makes it expand more. NAMOA* is the basis of our Tier 1 engine. Its exactness is the reason we

chose it over a population method: the front is the same on every run, and interior points that a payload might need are never lost to chance.

2.1.6 Payload-mass-aware trajectory planning

Tsoutsouras et al. [21] look at drones shared by several users, where packages of different mass are dropped at different waypoints before their deadlines. They build a power model in which propulsion power depends on both payload mass and flight speed, and formulate trajectory planning as minimising deadline misses under energy, waypoint and mass constraints, a problem close to the travelling salesman problem. A greedy heuristic produces a first trajectory and an optimiser improves it, and the whole computation runs on the drone, in under about 320 ms on an ARM Cortex-A9.

This is the closest work to ours in that payload matters to the route. The way it matters is different, though. Mass is a continuous physical parameter inside the energy and speed model, and the planner returns one trajectory. There is no Pareto front, and no discrete payload description such as fragility or urgency that chooses between trade-offs.

2.1.7 Quality indicators for Pareto fronts

Zitzler and Thiele [22] introduced the size of the dominated space, now called the hypervolume (HV): the volume of objective space a front dominates up to a reference point. It rewards both closeness and spread, and it is the main front-quality measure in our benchmarks.

2.1.8 Effect of drone transport on medical payloads

Three studies measure what drone flight does to medical cargo. Oakey et al. [14] recorded vibration on fixed-wing and multi-copter drones and found levels up to three times those of road transport, at higher frequencies, but insulin carried on those flights showed no aggregation. Theobald et al. [20] flew monoclonal antibody infusions on sixty flights with vibration between 1.5 and 3 g and found no change in particle size or aggregation. Stierlin et al. [19] compared drone and car transport of blood samples and found them comparable, with small differences in five analytes.

Taken together, these results say the tested payloads survive ordinary flights. They do not say what happens to a more fragile cargo, or whether a calmer corridor would help, because none of the studies changed the route. We read this as a reason to treat fragility as an operator's precaution in the planner, not as a proven damage model.

2.1.9 Summary of the reviewed work

Table 2.1 summarises the reviewed papers.

TABLE 2.1: Summary of the reviewed work

No.	Author / Year	Method	Findings	Limitation for our problem
1	Chokkalingam and Ganesan (2026) [4]	POPSO with hybrid initialisation, adaptive swarm parameters and Pareto-entropy archive	Well-distributed fronts; better path efficiency, safety and adaptability than compared methods	Final route chosen by a weighted sum with empirically set weights; wind enters as a speed-only factor; payload absent
2	Ren et al. (2019) [18]	CDNSGA-II over an octree city model with a safety index map; A* seeding	Spread-out, collision-free paths for distance and safety	Selection by crowding distance only; no mission context; no wind
3	Deng et al. (2026) [7]	MS-INSGA2 with safe interpolated seeding, adaptive operators and local search	Beats eleven algorithms on 3-D tasks; better benchmark indicators	No payload or delivery context; wind left to future work; front rebuilt per task
4	Deb et al. (2002) [6]	NSGA-II: fast non-dominated sorting, elitism, crowding distance	Better spread and convergence than PAES and SPEA on test problems	Crowding distance is purely geometric
5	Coello Coello et al. (2004) [5]	MOPSO with external repository, adaptive grid and mutation	Competitive fronts at low cost	Stochastic; may miss interior points of the front
6	Mandow and Pérez de la Cruz (2010) [11]	NAMOA*: label-based multi-objective A*	Exact front with admissible heuristics; efficiency results under consistent heuristics	Says nothing about which route to fly
7	Tsoutsouras et al. (2020) [21]	Mass- and speed-aware power model; heuristic plus optimiser for deadline-constrained multi-drop trajectories	Near-optimal trajectories on board in about 320 ms	One trajectory, no front; mass is a physical parameter, not a selector
8	Zitzler and Thiele (1999) [22]	Comparative study of MOEAs; SPEA; dominated-space indicator	Hypervolume as a front-quality measure	Evaluation tool only

No.	Author / Year	Method	Findings	Limitation for our problem
9	Oakey et al. (2021); Theobald et al. (2023); Stierlin et al. (2024) [14, 19, 20]	Vibration and sample-quality measurements on drone flights	Tested medicines and blood samples unaffected by ordinary flights	Route never varied; no evidence for or against route-based protection

2.2 GAPS IDENTIFIED

Two of the planners we reviewed make the diversity of the front their central design concern, and only one of those two uses it to select a route. The crowding distance of Ren et al. [18] picks a point directly from the front. Chokkalingam and Ganesan [4] use entropy only to keep the archive diverse during optimisation, and fall back on a weighted sum, with weights set per mission, to make the final pick. Deng et al. [7] improve the front itself and leave the choice open. Tsoutsouras et al. [21] do not build a front at all: they put continuous payload mass into the energy model and adapt speed as mass changes, but never choose between competing trade-offs by payload.

So none of these works lets a payload property act as an input to a Pareto-point selection step. This follows from how the mechanisms are defined: entropy and crowding distance are functions of the front's geometry, and mass-aware adaptation is a function of the physical model. Adding payload information would mean replacing the selection rule outright, which is what our Tier 2 does.

We state the gaps as follows.

Gap 1. No payload-aware route selection. Existing UAV multi-objective planners apply a fixed geometric criterion that ignores the payload, a hand-tuned weighted sum, or no selection mechanism at all.

Gap 2. No reuse of one front across mission types. Existing planners re-run the full multi-objective search whenever the mission changes. Nothing separates building the front, which depends on the region and the weather, from choosing a route, which depends on the payload.

Gap 3. No algebraic bridge from payload to trade-off. No explicit, reproducible mapping exists from a payload description to the routing trade-off it implies (fragility to low risk, urgency to short distance, weight to low energy).

Gap 4. Wind direction is not resolved per segment. Where weather appears at all, it is a speed-only factor applied uniformly [4]. A headwind and a tailwind on the same leg cost the same, and a stored front carries no record of the weather it was built for, so there is no rule for when it goes stale.

2.3 RESEARCH OBJECTIVES

The objectives of this project follow from these gaps.

1. To formulate multi-objective UAV route planning as a Pareto front precomputation task over a delivery airspace graph, and to justify an exact, deterministic graph search over stochastic population methods for this task.

2. To build a three-dimensional airspace lattice from terrain, buildings, restricted zones, water and land use, with wind-aware, direction-dependent

energy and risk on every edge, and a rule for when a stored front must be rebuilt.

3. To design a payload metadata schema and an explicit formula that turns mission context (fragility class, payload weight class, delivery urgency, priority) into Pareto-point selection weights with no expert tuning.

4. To benchmark Tier 1 front quality against single-objective A* and MOPSO on the same scenarios, using hypervolume, inverted generational distance and spread.

5. To gather evidence that Tier 2 picks routes that match the payload, including non-extreme routes for mixed payloads, and does so within one second.

6. To export the selected route as an autopilot mission and verify it in a Gazebo and ArduPilot SITL simulation before hardware flights in Phase II.

Table 2.2 links each gap to its objectives.

TABLE 2.2: Gaps identified and the research objectives that address them

Gap	**Objectives**	**How the system addresses it**
Gap 1: no payload-aware selection	3, 5	Tier 2 derives weights from four payload tags and scores every route on the front; argmax gives the mission route
Gap 2: no front reuse	1, 4	Tier 1 builds an exact NAMOA* front once per region and weather window; a front store reuses it for every mission until it expires or the weather drifts
Gap 3: no algebraic bridge	3, 5	Weight formula $w_{\text{risk}} \propto 2F$, $w_{\text{dist}} \propto 3U$, $w_{\text{energy}} \propto 2W$, with priority amplification; deterministic and checked for monotonicity over all 225 tag combinations
Gap 4: wind direction not resolved	2, 6	Headwind and crosswind computed per edge from its bearing; gust, turbulence and a go/no-go gate; front stored with its weather snapshot and refreshed on drift

CHAPTER 3

PROPOSED METHODOLOGY

The whole system rests on one design decision: Pareto front construction (Tier 1) is kept apart from route selection (Tier 2). The front depends on the region and the weather, so it is built once per delivery region and refreshed when it goes stale. The choice of route depends on the payload, so it is made per mission, against whatever front is current. A mission that arrives between refreshes runs only Tier 2, which reads the payload tags and returns a route in a fraction of a millisecond.

The workflow is as follows. Environment data for the region is fetched and cached. The airspace graph is built from it, with rules for which nodes and edges are allowed and a shared pipeline that costs every edge. Tier 1 computes the Pareto front and stores it. Tier 2 turns each payload tag into a route, and the route is exported as an autopilot mission and flown in simulation. Baseline planners and an evaluation module score each stage, and what they find goes back into the model. Figure 3.1 gives an overview.

3.1 ARCHITECTURE DESIGN

Figure 3.2 shows the system architecture, which has five layers. The environment data layer fetches weather, terrain and OpenStreetMap layers and caches them on disk, so that a network failure falls back to the last good copy instead of stopping the planner. Module 1 turns these layers into a directed airspace graph whose edges carry a three-objective cost vector. Tier 1 (Module 2) computes the Pareto

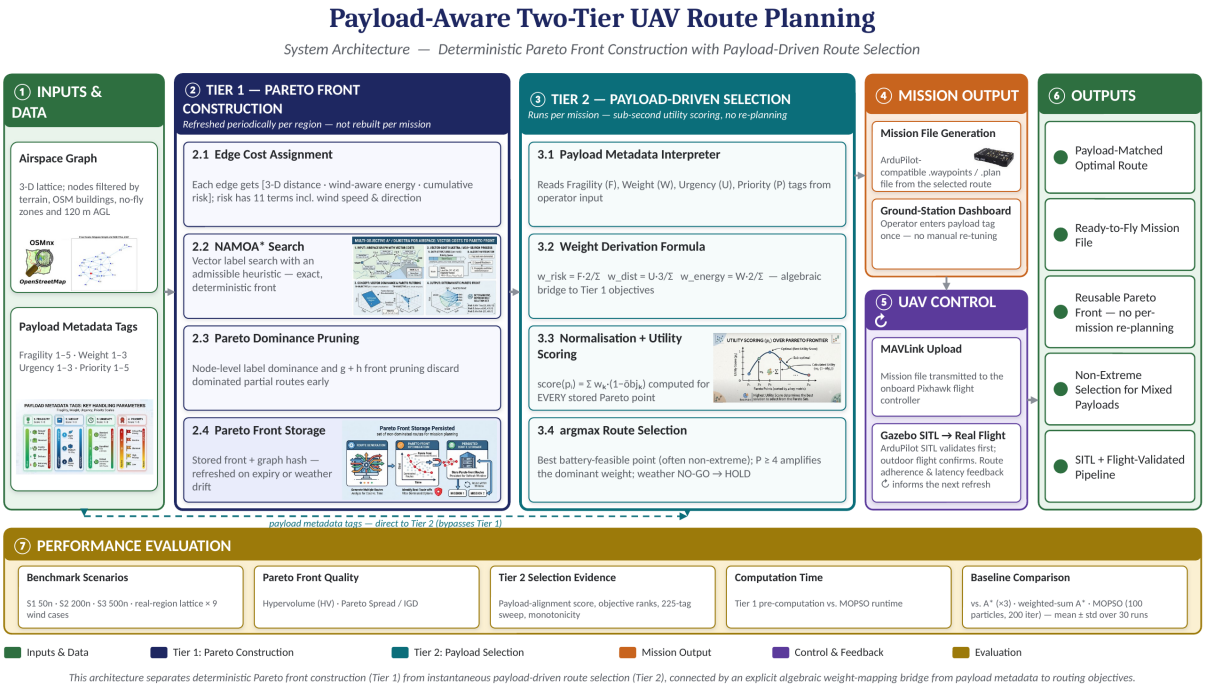


FIGURE 3.1: Methodology overview: Tier 1 Pareto front construction feeding Tier 2 payload-aware route selection, with mission output, UAV control and evaluation

front over that graph and keeps it in a front store, together with what is needed to decide when it is stale. Tier 2 (Module 4) reads the operator's payload tags and returns one route from the stored front, or a HOLD advisory when the weather or the battery rules the mission out. The output layer (Modules 6 and 7) writes the mission file and flies it in Gazebo with ArduPilot SITL. Modules 3 and 5 sit beside the main path. They run the baseline planners and compute the quality measures used in Chapter 4.

Every edge $e = (u, v)$ of the airspace graph carries the cost vector

$$\mathbf{c}(e;\omega) = \left[d(e),\ \varepsilon(e;\omega),\ \rho(e;\omega)\right] \tag{3.1}$$

where $d(e)$ is the three-dimensional length of the segment in kilometres, $\varepsilon(e;\omega)$ is the wind- and grade-aware energy proxy and $\rho(e;\omega)$ is the cumulative risk

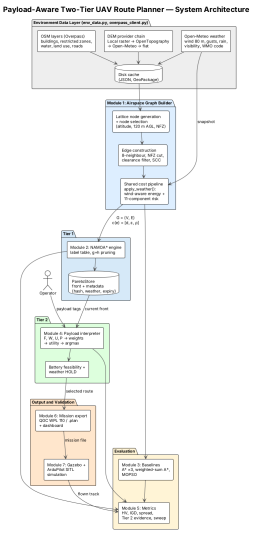


FIGURE 3.2: System architecture. Tier 1 builds and stores the front per region and weather window; Tier 2 selects a route per payload tag; Modules 6 and 7 export and fly it

exposure along the segment under weather snapshot ω. Section 3.2 defines all three. The three objectives are to minimise distance, energy and risk. Route p dominates route q, written $p \prec q$, when p is no worse than q on all three objectives and strictly better on at least one. The Pareto front $\mathcal{P}$ is the set of routes that no other route dominates.

The code is organised into seven modules. The numbering follows the source files, so Module 3 holds the baselines and Module 4 holds Tier 2. The sections below describe each module's purpose, inputs, processing and outputs.

3.2 AIRSPACE GRAPH BUILDER (MODULE 1)

Module 1 builds the delivery airspace graph $G = (V, E)$. Its inputs are the region (centre, box size and lattice spacing), the environment layers described below and a weather snapshot ω. Its output is a directed NetworkX [8] graph whose nodes carry position and altitude and whose edges carry static features, a per-component

risk breakdown and the cost vector $[d, \varepsilon, \rho]$. The weather go/no-go state is stored on the graph itself so that Tier 2 can read it. Two builders share one cost pipeline: a lattice builder over a real region, described below, and a seeded synthetic builder for controlled benchmarks (Section 4.1).

3.2.1 Environment data acquisition

Weather comes from the Open-Meteo forecast service [15]: the 10 m wind, gust, rain, visibility and WMO code, and the hourly 80 m wind speed and direction. With no network the last cached snapshot is used, and with no cache the planner assumes calm air so that runs stay deterministic. Terrain comes from a chain of elevation providers tried in order (a local GeoTIFF, OpenTopography Copernicus 30 m, Open-Meteo elevation, and flat ground as a last resort), each cached on disk. OpenStreetMap buildings, restricted zones, water and land use are fetched through a multi-endpoint Overpass client with retries, cached as GeoPackage files [3] and indexed in a local metric frame with Shapely STRtree structures for fast height, distance, containment and segment-intersection queries.

3.2.2 Lattice generation and node selection

A UAV is not tied to roads, so the airspace is a regular lattice, not a road network. The region is covered by an $n \times n$ lattice in a local equirectangular frame centred on the region, with $n = \text{round}(box/spacing)$. The default 6 km box at 200 m spacing gives 900 candidate nodes. A candidate does not enter the graph automatically. It has to pass a selection rule that combines four data sources: ground elevation from

the DEM, the tallest building in the node's cell, the legal ceiling and restricted airspace. For node k with ground elevation z_k and tallest building b_k in its cell,

$$\mathrm{AGL}_k = \max\bigl(H_{\mathrm{cruise}},\, b_k + C\bigr), \qquad \mathrm{alt}_k = z_k + \mathrm{AGL}_k \tag{3.2}$$

with cruise height $H_{\mathrm{cruise}} = 60\,\mathrm{m}$ and clearance $C = 15\,\mathrm{m}$. A building's height is read from its `height` tag, else from `building:levels` times 3 m, else from a default for its type. A node with $\mathrm{AGL}_k > 120\,\mathrm{m}$ is blocked, because it cannot be flown legally under the Drone Rules, 2021 [13]. A node inside a restricted polygon (an aerodrome, military land, a prison, a protected area or a government site) is removed. Every surviving node stores its position in both frames, its ground elevation, the building height, density and mean height of its cell, its height above ground and its altitude above mean sea level.

3.2.3 Edge construction and pruning

Each surviving node is linked to its eight neighbours in both directions, so the graph is directed from the start. Three filters then remove edges. The first drops any edge whose straight segment crosses a restricted polygon, which stops a route from clipping the corner of a zone between two legal nodes. The second is an optional hook, `edge_filter`, which the simulation module uses for hard roof and terrain clearance. The third keeps only the largest strongly connected component, so that every remaining node can reach every other.

For each edge that survives, the module stores the horizontal haversine length d_h, the forward-azimuth bearing θ, the grade $g = \Delta\mathrm{alt}/d_h$ and the three-dimensional

length $d = \sqrt{d_h^2 + \Delta\text{alt}^2}$. The bearing is

$$\theta = \operatorname{atan2}\big(\sin\Delta\lambda\cos\varphi_2,\ \cos\varphi_1\sin\varphi_2 - \sin\varphi_1\cos\varphi_2\cos\Delta\lambda\big) \tag{3.3}$$

where φ is latitude and λ is longitude, so that the east component is correctly scaled by $\cos\varphi$.

3.2.4 Static risk terms

Static terms do not depend on the weather, so they are computed once per build. Each edge is sampled every 25 m along the straight line between its two endpoint altitudes. At each sample the module knows the flight altitude, the ground height, the tallest roof directly below and the tallest roof within 30 m. For the default region this means roughly 6,800 directed edges, about 75,000 sample points and around 375,000 spatial-index queries per build. Table 3.1 lists all eleven risk components, static and weather-dependent, with the weight each carries in the edge score

$$\rho_{\text{edge}} = \operatorname{clip}\Big(\sum_k w_k\,\rho_k,\ 0,\ 1\Big), \qquad \rho(e;\omega) = \rho_{\text{edge}} \cdot \frac{d_h}{s} \tag{3.4}$$

where s is the lattice spacing. Multiplying by d_h/s turns the per-edge score into a cumulative exposure. A diagonal hop counts $\sqrt{2}$ times an axis hop, and the risk of a route grows with the distance flown through risky air.

TABLE 3.1: Risk components (every $\rho_k \in [0,1]$) and their weights

Component	**Model**	w_k
Obstacle	Seeded obstacle discs, $0.9(1-r/R)$ inside a disc (synthetic graphs only)	1.00
Building	Mean over samples of [roof within 30 m] $\cdot\big(1-\mathrm{clip}(m/60)\big)$, m = margin above that roof	0.60
Terrain	$\max\big(1-\mathrm{clip}(m_{\min}/15),\ \mathrm{clip}(\lvert g\rvert/0.30)\big)$: lowest clearance margin or sudden altitude change	0.30
Wind	$\mathrm{clip}(CW/40)$, crosswind against a 40 km/h saturation speed	0.30
Gust	$\mathrm{clip}\big(0.5\max(0, gust-v)/40\big)$	0.30
Weather	$0.5\,\mathrm{clip}(rain/7.5)+0.5\,\mathrm{clip}(1-vis/10\,\mathrm{km})$, uniform over the graph	0.20
Population	Mean land-use class score along the edge (our values, ranked in the spirit of SORA [10]): assembly 1.0, commercial 0.7, major road 0.6, residential 0.5, industrial 0.3	0.50
No-fly buffer	$\max_k \exp(-\delta_k/150\,\mathrm{m})$ to the nearest restricted zone, zero beyond 600 m	0.80
Water	Fraction of samples over water (risk of losing the payload)	0.30
Comm/GPS	$\mathrm{clip}(density/0.30)\cdot\mathrm{clip}(\bar{h}/30\,\mathrm{m})$, urban canyon effect	0.20
Turbulence	$\mathrm{clip}(density/0.30)\cdot\mathrm{clip}(v/40)$	0.20

The weights are design values that put restricted zones, buildings and crowds ahead of the weather terms. They have not been calibrated against flight data, and calibrating them is part of the evaluation work that remains in Phase I.

3.2.5 Wind-aware energy model

For an edge on bearing θ_{edge} and a wind of speed v blowing from θ_{from} (the meteorological convention), the headwind and crosswind components are

$$HW = v\cos(\theta_{\text{edge}} - \theta_{\text{from}}), \qquad CW = v\,|\sin(\theta_{\text{edge}} - \theta_{\text{from}})| \tag{3.5}$$

A positive HW is a headwind. The energy proxy of the edge is

$$\varepsilon = d\left(1 + |g| + \beta_h \frac{\max(0, HW)}{V_c} + \beta_c \frac{CW}{V_c}\right) \tag{3.6}$$

with cruise airspeed $V_c = 36$ km/h, $\beta_h = 0.40$ and $\beta_c = 0.15$. A tailwind earns no credit. This is deliberate: it keeps $\varepsilon \geq d$ on every edge, and the Tier 1 heuristic relies on that bound. The wind speed is the 80 m forecast when available, which is close to the 60–120 m cruise band. Otherwise the 10 m value is lifted to cruise height with the power law

$$v_h = v_{10}\left(\frac{h}{10}\right)^{\alpha}, \qquad \alpha = 0.25 \tag{3.7}$$

following Peterson and Hennessey [17]. A gust factor measured at 10 m is applied to the wind at flight height.

Because of Equation (3.5), edge costs are directional: flying $u \to v$ and $v \to u$ cost different amounts under wind. This is why the graph is directed even though the lattice itself is symmetric.

3.2.6 Weather-dependent risk and the go/no-go gate

Four of the eleven risk components change with the weather: crosswind, gust excess, turbulence and the uniform rain and visibility term. Crosswind depends on the edge heading, so the same wind makes some corridors riskier than others, and a change of direction alone can move the low-risk end of the front to a different part of the map. The function `apply_weather(G, ω)` re-costs every edge from its stored static features, so a new snapshot never requires rebuilding the spatial layers.

The weather gate sits on top of this. A mission is a no-go when the WMO code reports a thunderstorm (95–99), when rain reaches 7.5 mm/h, when visibility falls below 1 km, or when gusts exceed 45 km/h, a provisional limit for an F450-class airframe. The reasons are written to the graph, and Tier 2 refuses to select a route while any reason is present.

3.3 TIER 1 PARETO FRONT ENGINE (MODULE 2)

Module 2 computes the exact set of non-dominated routes between a source and a target. Its inputs are the costed graph from Module 1, the two end nodes and an expansion cap. Its output is the Pareto front $\mathcal{P}$, as a list of cost vectors and node paths, together with diagnostics: whether the cap truncated the search, how many labels were expanded and generated, and the runtime.

3.3.1 Labels and dominance

The engine follows NAMOA* [11]. Every partial path is a label $(node, \mathbf{g}, parent)$ held in a flat table, and a path is rebuilt at the end by walking parent ids. Nothing is copied when a label is pushed, which matters once the label count runs into the thousands. A new label $\mathbf{g}'$ at node v is discarded if any existing label at v weakly dominates it (is $\leq$ in every objective, within a small tolerance). Equal vectors count as dominated, so the front never contains duplicates. Labels that the new one dominates are marked dead and skipped when popped. No visited set is needed. Every edge has positive distance, so a path that revisits a node is dominated by its own earlier label at that node, and cycles die in the dominance check.

3.3.2 Admissible vector heuristic and pruning

The heuristic for node v is

$$\mathbf{h}(v) = [h,\ h,\ 0] \tag{3.8}$$

where h is the great-circle distance from v to the target. It is admissible for all three objectives. For distance, every edge has $d \geq d_h$, and the horizontal lengths of any path to the target add up to at least h. For energy, Equation (3.6) gives $\varepsilon \geq d$, so the same bound holds. For risk, $\rho \geq 0$. A label is pruned when the current front weakly dominates $\mathbf{g} + \mathbf{h}$. The check is made when the label is generated and again when it is popped, because the front may have improved in between. The open list is ordered by $\|\mathbf{g} + \mathbf{h}\|$, which tends to complete good routes early and so tightens the pruning sooner.

3.3.3 Truncation and front thinning

An expansion cap protects the planner from a runaway search. When the cap is hit, the result is flagged as truncated only if some live label could still improve the front. An optional ε-dominance step can cap very large fronts at N routes while keeping the three single-objective extremes; it is off by default, so the benchmarks use the exact front.

Algorithm 1 gives the full procedure. Here $\preceq$ is weak dominance.

Algorithm 1 NAMOA* with a label table (Tier 1)

Require: Directed graph $G = (V, E)$, source s, target t, costs $\mathbf{c}(u, v) = [d, \varepsilon, \rho]$, cap M

Ensure: Pareto front $\mathcal{P}$, flag *truncated*

$L \leftarrow [(s, [0,0,0], -1)]$; $at[s] \leftarrow \{0\}$; $F \leftarrow \emptyset$; $Open \leftarrow \{(\|\mathbf{h}(s)\|, 0)\}$
while $Open \neq \emptyset$ **do**
  if $expanded \geq M$ **then**
    $truncated \leftarrow \exists$ live $\ell \in Open$ with $F \not\preceq \mathbf{g}_\ell + \mathbf{h}(u_\ell)$; **break**
  end if
  $\ell = (u, \mathbf{g}) \leftarrow$ PopMin($Open$)
  if ℓ is dead **or** $F \preceq \mathbf{g} + \mathbf{h}(u)$ **then continue**
  end if
  $expanded \leftarrow expanded + 1$
  for each successor v of u **do**
    $\mathbf{g}' \leftarrow \mathbf{g} + \mathbf{c}(u, v)$
    if $\exists o \in at[v] : \mathbf{g}_o \preceq \mathbf{g}'$ **then continue** ▷ node dominance
    end if
    if $F \preceq \mathbf{g}' + \mathbf{h}(v)$ **then continue** ▷ prune on $g + h$
    end if
    mark dead every $o \in at[v]$ with $\mathbf{g}' \preceq \mathbf{g}_o$
    append label $(v, \mathbf{g}', \ell)$ to L and to $at[v]$
    if $v = t$ **then**
      $F \leftarrow \{f \in F : \mathbf{g}' \not\preceq f\} \cup \{\mathbf{g}'\}$
    else
      push $(\|\mathbf{g}' + \mathbf{h}(v)\|, \text{label})$ onto $Open$
    end if
  end for
end while
rebuild each route in F by following parent ids; **return** $\mathcal{P}$, *truncated*

3.3.4 Why an exact method

A population method is usually faster on a large graph, and its front can look similar by hypervolume. We still chose an exact search because the same graph and weather always give the same front, because the interior routes that mixed payloads need are the ones a stochastic archive tends to miss (Section 4.4.1), and because Tier 1 runs once per region and weather window, so a slower search is affordable.

3.3.5 Pareto front store and refresh

The front is reused across missions, so the system needs a rule for when reuse is safe. The front store saves one JSON record per (region, source, target) with the front, a SHA-256 hash of the static graph (never the weather-dependent costs), the weather snapshot it was built for, a validity limit three hours after creation, the truncation flag and the search diagnostics.

On each request the store lists the reasons the record cannot be reused : no record, a forced refresh, a different static graph, a different identity, expiry, or weather drift. Drift means the wind speed has moved by more than 10 km/h, the direction has turned by more than 45° while the wind is above 10 km/h, the gust has moved by more than 15 km/h, or the go/no-go state has flipped. With no reason, Tier 2 runs against the stored front and no search happens. With a reason, only `apply_weather` and Tier 1 run again, and the spatial layers stay cached.

3.4 BASELINE PLANNERS (MODULE 3)

Module 3 holds the planners that Tier 1 and Tier 2 are compared against. They read the same costed graph, so any difference in the results comes from the search method and not from the data.

Three single-objective A* searches minimise distance, energy and risk separately; the distance search uses the haversine heuristic and the other two a zero heuristic. Each returns one extreme point of the front. A weighted-sum A* scalarises the cost vector with a preset's Tier 2 weights, $c_w(e) = \sum_j w_j c_j(e)/s_j$, where s_j is the spread of objective j across the three extreme routes. This is the classic approach our problem statement criticises, and it can reach only supported points on the convex hull of the front. The population baseline adapts MOPSO [5] to graph routes: a particle is a valid path, the swarm is seeded with the extreme routes and biased random walks, and in each iteration a particle follows its personal best or an archive leader for part of the way and is repaired to the target with a weighted shortest suffix. The archive uses the same dominance helper as Tier 1. Report-scale settings are 100 particles, 200 iterations, an archive of 100 and 30 seeded repetitions.

3.5 TIER 2 PAYLOAD METADATA INTERPRETER (MODULE 4)

Module 4 turns a description of the payload into a route. Its inputs are the stored front, the weather go/no-go reasons from the graph and the four tags $(F, W, U, P_{\text{lvl}})$.

Its output is a selection record: the tags, the derived weights, the selected route's index, cost vector and waypoints (latitude, longitude and altitude), the normalised scores, the selected route's rank on each objective, whether it is an extreme or non-extreme point, the latency, the number of battery-feasible routes and, when no route can be flown, a HOLD advisory. The record is written to JSON and CSV. Each tag must be an integer in its range, and a bad tag raises an error before any weight is derived, so a typing mistake at the ground station cannot quietly change the route.

3.5.1 Semantic mapping

The bridge between the tiers maps each payload tag to exactly one Tier 1 objective, as Table 3.2 shows.

TABLE 3.2: Semantic mapping from payload tags to Tier 1 objectives

Payload tag	**Range**	**Tier 1 objective**	**Reasoning**
Fragility class F	1–5	w_{risk} (flight risk)	Fragile items should avoid turbulent, exposed and crowded corridors
Payload weight class W	1–3	w_{energy} (energy)	A heavier payload raises hover and lift power, so energy matters more
Delivery urgency U	1–3	w_{dist} (distance)	Time-critical deliveries need the shortest path
Priority level P_{lvl}	1–5	Amplifier	Strengthens the dominant weight for high-priority missions

3.5.2 Weight derivation

From the tags, three weights are derived with no expert input:

$$r_{\text{raw}} = 2F, \qquad d_{\text{raw}} = 3U, \qquad e_{\text{raw}} = 2W, \qquad S = r_{\text{raw}} + d_{\text{raw}} + e_{\text{raw}} \tag{3.9}$$

$$w_{\text{risk}} = \frac{r_{\text{raw}}}{S}, \qquad w_{\text{dist}} = \frac{d_{\text{raw}}}{S}, \qquad w_{\text{energy}} = \frac{e_{\text{raw}}}{S} \tag{3.10}$$

The multipliers (2 for fragility and payload weight, 3 for urgency) encode the assumption that, in medical delivery, urgency is more sensitive than fragility or payload weight. Payload weight is tied to energy because a heavier payload directly raises the power needed to hover and climb. No such direct physical link exists to risk or to distance.

Every weight depends on all three raw scores. Raising F raises w_{risk} and lowers the other two at the same time, so one tag change moves the whole weight vector. This is the reason the formula is not a lookup table. There is no fixed list of payload types, and any of the $5 \times 3 \times 3 \times 5 = 225$ tag combinations gives a valid, different weighting.

For high-priority missions ($P_{\text{lvl}} \geq 4$), the largest weight is amplified:

$$w_{\max} \leftarrow w_{\max} + 0.1\, w_{\max}, \qquad w_j \leftarrow \frac{w_j}{\sum_k w_k} \tag{3.11}$$

A flat bonus added to every score was rejected, because it never changes which route scores highest.

3.5.3 Utility scoring and selection

For each Pareto point p_i with objective values $(d_i, \varepsilon_i, \rho_i)$, the values are min-max normalised over the whole front,

$$\tilde{d_i} = \frac{d_i - d_{\min}}{d_{\max} - d_{\min}}, \quad \tilde{\varepsilon}_i = \frac{\varepsilon_i - \varepsilon_{\min}}{\varepsilon_{\max} - \varepsilon_{\min}}, \quad \tilde{\rho}_i = \frac{\rho_i - \rho_{\min}}{\rho_{\max} - \rho_{\min}} \tag{3.12}$$

so that 0 is the best value on the front and 1 the worst. A degenerate objective, where all routes are equal, normalises to zero. The utility score is

$$\text{score}(p_i) = w_{\text{dist}}(1 - \tilde{d_i}) + w_{\text{energy}}(1 - \tilde{\varepsilon}_i) + w_{\text{risk}}(1 - \tilde{\rho}_i) \tag{3.13}$$

and the mission route is $p^* = \arg\max_{p_i \in \mathcal{P}} \text{score}(p_i)$. Ties go to the lower index, so the result is deterministic.

The argmax runs over the full front, so Tier 2 can land anywhere on it, including between the extremes. For a payload with mixed needs the best-scoring route is often an interior one that no single-objective search would return.

3.5.4 Reference presets

Table 3.3 gives the weights the formula produces for four reference presets. Only emergency medicine has $P_{\text{lvl}} \geq 4$, so it is the only preset whose dominant weight is amplified.

As a worked example, take emergency medicine. The raw scores are $r_{\text{raw}} = 2$, $d_{\text{raw}} = 9$ and $e_{\text{raw}} = 2$, which sum to 13, so the weights before amplification are 0.154, 0.692 and 0.154. The priority is 5, so the distance weight grows by a tenth to

TABLE 3.3: Reference payload presets and their derived weights

Preset	F	W	U	P_{lvl}	w_{risk}	w_{dist}	w_{energy}
Fragile medical sample	5	1	1	3	0.667	0.200	0.133
Emergency medicine	1	1	3	5	0.144	0.712	0.144
Disaster relief	1	3	1	2	0.182	0.273	0.545
Mixed cargo	3	2	2	3	0.375	0.375	0.250

0.762. After renormalising by the new total of 1.069, the weights are $w_{\text{risk}} = 0.144$, $w_{\text{dist}} = 0.712$ and $w_{\text{energy}} = 0.144$, which are the values in the table.

3.5.5 Battery feasibility

The payload weight class also decides whether a route can be flown at all. Each class maps to a nominal payload mass m_W (0.25, 0.5 and 1.0 kg for $W = 1,2,3$), and a route's energy proxy is converted to watt-hours with

$$E_{\text{Wh}} = k_{\text{cal}}\,\varepsilon \left(\frac{m_{\text{uav}} + m_W}{m_{\text{uav}}} \right)^{1.5} \tag{3.14}$$

where the exponent follows the way induced rotor power scales with weight. With the current values ($k_{\text{cal}} = 6$ Wh per unit of energy proxy, $m_{\text{uav}} = 1.5$ kg, a 74 Wh battery and a 20% reserve), a route is feasible when it needs at most 59.2 Wh. Infeasible routes are removed before the argmax. These constants are marked as placeholders in the code and will be replaced by values calibrated in simulation.

3.5.6 Weather hold and decision record

If the weather gate reports any reason, or if no route on the front is battery-feasible, Tier 2 returns a HOLD advisory that says why, instead of a route. Otherwise it

returns the feasible route with the highest score.

3.6 EVALUATION AND BENCHMARKING (MODULE 5)

Module 5 measures how good a front is and how sensible the Tier 2 picks are. Every measure is computed per scenario, and the results are written to CSV and JSON so that each pass of the evaluation can be compared with the previous one.

3.6.1 Front quality measures

There is no known true front for our graphs, so the reference front $\mathcal{R}$ of a scenario is the non-dominated union of every route found by every method and repetition, and all measures are computed in the space normalised by its ideal and nadir points. The hypervolume (HV) [22] is the volume dominated by a front and bounded by a reference point, here the normalised nadir padded by 10%, $[1.1, 1.1, 1.1]$. It is computed exactly in three dimensions by slicing along the first objective and solving the two-dimensional problem on each slice. Larger is better. The inverted generational distance (IGD) measures how well a front A covers the reference front:

$$\mathrm{IGD}(A, \mathcal{R}) = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} \min_{a \in A} \|r - a\|_2 \tag{3.15}$$

Smaller is better, and zero means that A contains every reference point. The spread Δ measures how evenly the points are distributed and how far they reach toward

the extremes of $\mathcal{R}$; smaller is better. Runtime and front size are recorded for every method.

3.6.2 Tier 2 evidence

We do not report a Tier 2 "accuracy", because there is no ground-truth label that says which route is correct for a payload. For each preset the module records the selected route's rank on each objective, whether it is an extreme or non-extreme point, the latency, and a rank-based payload-alignment score

$$A = \sum_{k \in \{d,\varepsilon,\rho\}} w_k \left(1 - \frac{\text{rank}_k - 1}{K - 1}\right) \tag{3.16}$$

where K is the front size. Because A uses ranks, it does not depend on the min-max utility that Tier 2 itself maximises. The same score is computed for four competing selection rules on the same front: the shortest route, the knee point (the point farthest from the line joining the distance and risk extremes), the largest NSGA-II crowding distance [6, 18], and a fixed equal-weight utility in the style of [4].

3.6.3 Tag sweep and monotonicity

A sweep runs Tier 2 for all 225 tag combinations. It counts distinct selected routes and non-extreme picks, and it checks monotonicity: raising F with the other tags fixed must never raise the selected route's risk, raising U must never raise its distance, and raising W must never raise its energy. The target is zero violations.

3.6.4 The evaluation cycle

Benchmarking is a repeated cycle, not a single run: fix the scenarios, run every method, score the fronts, check the Tier 2 picks, fly the routes in simulation, analyse the weak spots, change the model and run everything again. One full pass covers four graphs under nine wind cases with MOPSO at 100 particles, 200 iterations and 30 repetitions, about 21.6 million particle updates in all.

3.7 MISSION EXPORT AND DASHBOARD (MODULE 6)

3.7.1 Mission file generation

Module 6 writes the selected route as a QGC WPL 110 file and a QGroundControl `.plan` file, merging consecutive waypoints that keep the same bearing and slope. Route waypoints use frame 3 (`GLOBAL_RELATIVE_ALT`), measured from home, so the exporter writes

$$alt_{\text{rel}} = alt_{\text{AMSL}} - H_{\text{home}} \tag{3.17}$$

with H_{home} the ground elevation under the first waypoint. The height above the local ground is never written into frame 3, because that would be wrong wherever the terrain varies.

3.7.2 Operator dashboard

A small Tkinter dashboard lets an operator load a stored graph and front, type the four tags, see the route on a map and export the mission. Two defects are open: it does not yet pass the weather reasons to Tier 2, and it can load a graph and a front from different runs.

3.8 SIMULATION AND SITL VALIDATION (MODULE 7)

A route that scores well on the graph may still be hard to fly, so Module 7 checks the flight itself. It flies the Tier 2 route through a three-dimensional model of the delivery area in Gazebo Harmonic [16], with ArduCopter running in software-in-the-loop [1], and compares the flown track with the plan. Communication with the autopilot uses MAVLink [12]. The simulation code imports the planner modules directly; nothing is copied. Figure 3.3 shows the pipeline.

3.8.1 Bundle, clearance and frame contract

A region is first frozen into a bundle: a terrain grid, OSM buildings with heights, roads, the depot and destination pads, and a manifest. Its hash is written into the graph, the front, the mission file and the world, and the runner refuses to fly if they do not match. Before NAMOA* runs, a clearance checker passed in as `edge_filter` removes every edge that passes less than 15 m above a buffered roof or the terrain, or goes above 120 m above ground; waypoints are never raised after

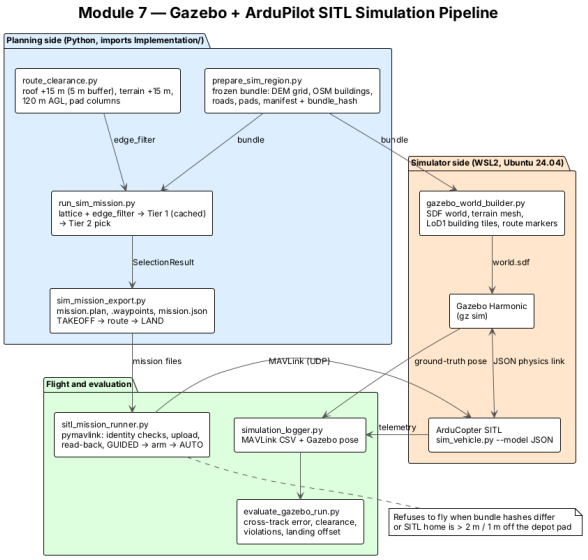


FIGURE 3.3: Gazebo and ArduPilot SITL simulation pipeline

selection. The world builder writes a Gazebo SDF world with a terrain mesh, LoD1 buildings, roads, pads and route markers, with its origin at the depot pad, where SITL also places home. The mission altitude therefore means the same height in the planner, the world and the autopilot. The runner, built on pymavlink, uploads the mission (take-off, route, landing on the pad), reads it back and compares it, arms, switches to AUTO and logs telemetry until the vehicle disarms.

An evaluator then computes the cross-track error against the planned polyline, the minimum clearance, any rule breach and the landing offset from the pad.

CHAPTER 4

EXPERIMENTAL RESULTS AND PERFORMANCE ANALYSIS

This chapter describes the data the system runs on, the hardware and software used, the experiments carried out so far and what their results show. The numbers come from the first pass of the evaluation cycle in Section 3.6, from a controlled wind experiment, from the ablation runner and from the first SITL flights. They are preliminary in a specific sense: MOPSO was run at reduced settings, the risk weights and battery constants are not yet calibrated, and the synthetic density labels have a known defect. We state each caveat next to the result it affects, so that the reader can judge how far the result can be pushed.

4.1 DATASET DESCRIPTION

Route planning papers rarely publish their environments. The datasets of the works we reviewed are custom, proprietary or not released, so a direct numerical comparison on their data is not possible. As is common in this field, we built our own environments. There are two kinds: synthetic airspace graphs, generated from fixed seeds, for controlled benchmarks; and a real-region lattice built from public geographic and weather data. Table 4.1 lists every graph used in this chapter.

TABLE 4.1: Graphs and scenarios used in the experiments

Scenario	Size	Description
S1 small sparse	50 nodes	Synthetic grid, seed 101, 5% obstacle coverage label, weather seeded
S2 medium	196–200 nodes	Synthetic grid, seed 202, 20% label; weather seeded (benchmark) or set explicitly (wind experiment)
S3 large dense	500 nodes	Synthetic grid, seed 303, 40% label, weather seeded
Real-region lattice	893 nodes, 6,752 edges	6 km box, 200 m spacing, offline fixture layers (ablation)
Small lattice	2 km box, 200 m spacing	Lattice used for the route figure in Section 4.4.5
Flat lattice	225 nodes, 1,624 edges	3 km box, 200 m spacing, no layers (directional cost check)
Simulation bundle	1.5 km box, 100 m lattice	Frozen terrain, buildings, roads and pads for Gazebo and SITL

4.1.1 Synthetic airspace graphs

The synthetic graphs come from the seeded builder of Section 3.2. Each is a jittered grid of about 1.3 km spacing with 8-neighbour directed edges, seeded Gaussian hills for terrain, obstacle discs, and seeded wind between 12 and 28 km/h with a gust factor of 1.4. The three scales (50, 200 and 500 nodes) were chosen to cover a sanity check, a mid-sized core case and a stress case. Because every draw comes from the seed, anyone can regenerate the exact graph from the seed and the code.

There is one known defect in this data. The obstacle density labels (5%, 20% and 40%) do not reach the graph builder. The benchmark runner scales the final risk instead, so the three scenarios differ in size and seed, but not in obstacle coverage as the labels suggest. Fixing this is part of the next pass, and every benchmark number will be recomputed after it.

4.1.2 Real-region lattice

The real-region lattice is built from four public data sources, listed in Table 4.2. The method is region-agnostic; we use one coastal urban neighbourhood as the example region. The default 6 km box at 200 m spacing gives 900 candidate nodes, of which 893 remain after node selection and connectivity pruning in the fixture run, linked by 6,752 directed edges.

TABLE 4.2: Environment data sources for the real-region lattice

Layer	**Source**	**Used for**
Weather	Open-Meteo forecast [15]	Wind speed and direction at 10 m and 80 m, gust, rain, visibility, WMO code
Terrain	Local GeoTIFF, OpenTopography Copernicus 30 m, Open-Meteo elevation (Copernicus 90 m), flat fallback	Ground elevation of nodes and edge samples
Buildings	OpenStreetMap via Overpass	Footprints and heights; node altitude, building risk, density, turbulence, urban canyon
Restricted zones	OpenStreetMap via Overpass	Aerodromes, military land, prisons, protected areas, government sites; node and edge removal, soft buffer
Water and land use	OpenStreetMap via Overpass, coastline	Water fraction, population exposure classes, major roads

Public Overpass servers are not always reliable. Live fetches sometimes reset or time out, and a failed fetch falls back to empty layers while the run still completes. For repeatable offline runs the project therefore includes fixture layers, hand-made test data shaped like the real region. The ablation in Section 4.4.4 used these fixtures. They show how each layer acts on the planner, but they are not real-world measurements, and the live-data run is part of the next pass.

4.1.3 Payload tag space and simulation bundle

Tier 2 is evaluated over all 225 tag combinations, with four reference presets (Table 3.3) that stand for risk-dominated, distance-dominated, energy-dominated and balanced deliveries. For simulation, a 1.5 km region is frozen into a bundle whose hash ties the planner graph, the stored front, the mission file and the Gazebo world together.

4.2 ECOSYSTEM

4.2.1 Hardware configuration

All planner experiments in this chapter ran on a laptop-class CPU without a GPU. The planner does not need one: Tier 1 is a graph search that runs on a single core, and Tier 2 is a closed-form computation. The simulation runs under WSL2 (Ubuntu 24.04) on the same class of machine.

Phase II moves the system onto standard, commercially available hardware: an F450 quadcopter frame with a Pixhawk flight controller running ArduCopter, a Raspberry Pi 5 companion computer that hosts Tier 2 on board, a servo payload release and a MAVLink telemetry link to the ground station.

4.2.2 Software environment

All code is written in Python 3. NetworkX [8] holds the directed airspace graph and runs the A* baselines; NumPy handles vectorised dominance checks and

metrics; Shapely 2 STRtree indexes hold buildings, zones, water and population classes; GeoPandas, pyogrio and OSMnx [3] parse and cache the OpenStreetMap layers; and the requests library calls Open-Meteo, OpenTopography and Overpass. Figures are drawn with Matplotlib, the operator dashboard uses Tkinter, and the test suites use unittest and run offline without keys. Simulation uses Gazebo Harmonic [16], ArduPilot SITL [1], pymavlink over MAVLink [12] and QGroundControl. Diagrams were drawn in PlantUML and the report typeset in LaTeX.

4.3 EXPERIMENTS CONDUCTED

Table 4.3 summarises the experiments. Each is described below, and Section 4.4 analyses the results.

The settings were as follows. In E1, all methods used the same source and target per scenario, and HV, IGD and spread were computed against the union reference front. MOPSO ran at a reduced setting of 24 particles, 30 iterations and 2 repetitions per scenario to get a complete first pass quickly; the report-scale run (100, 200 and 30) is the next step. In E2, the S2 graph (196 nodes, 1,404 edges, seed 202) was costed under calm air and under a 25 km/h wind with 35 km/h gusts from each of eight compass directions, then in 15° steps, with the graph, layers, source and target unchanged. E3 used a flat 3 km lattice with no terrain, buildings or zones under a 25 km/h east wind. E4 rebuilt the real-region front with all layers, without wind, without terrain and without buildings, under a 30 km/h wind from the south-west. E5 ran the four presets on the S2 front (16 routes), the S3 front (115 routes) and a 2 km lattice. E6 flew planner missions

TABLE 4.3: Experiments carried out in Phase I so far

No.	Experiment	Data	Question
E1	Front quality benchmark	S1, S2, S3	Does the exact Tier 1 front beat single-objective A* and MOPSO on HV, IGD and spread, and at what runtime cost?
E2	Wind direction study	S2 under calm air and 8 directions; 15° sweep	How much does wind direction alone change the front and the Tier 2 picks?
E3	Directional edge costs	Flat lattice, wind from the east	Do edge costs respond to heading as the model predicts?
E4	Risk-layer ablation	Real-region lattice, fixture layers	Which layers drive risk, energy and front size?
E5	Tier 2 selection	S2, S3 and small lattice fronts	Do different payloads get different routes, including non-extreme ones, within 1 s?
E6	SITL flights	Fixture bundle, ArduCopter SITL	Is the mission chain from Tier 2 to the autopilot correct?
E7	Automated tests	Planner and simulation suites	Do the components behave as specified?

from a fixture bundle against ArduCopter SITL (4.8.0-dev, built-in quadcopter physics, five times real time), and a Tier 2 mission was also flown in the Gazebo city world. E7 covers about 220 planner tests and 45 simulation tests, all offline; Appendix B lists them. One of them compares the NAMOA* front with a brute-force front over all simple paths on a small graph, which is our strongest check that Tier 1 is exact.

4.4 PERFORMANCE ANALYSIS

4.4.1 Tier 1 front quality against the baselines

Table 4.4 gives the first-pass benchmark.

TABLE 4.4: First-pass benchmark on the synthetic scenarios (HV higher is better; IGD and Δ lower are better)

Scenario	Method	Front	Time (s)	HV	IGD	Δ
S1 (50)	NAMOA* (Tier 1)	2	0.003	0.131	0.000	0.000
	A* distance	1	<0.001	0.121	0.866	–
	A* energy	1	<0.001	0.121	0.866	–
	A* risk	1	<0.001	0.011	0.866	–
	MOPSO	2	0.363	0.131	0.000	0.000
S2 (200)	NAMOA* (Tier 1)	16	0.162	1.2065	0.000	1.001
	A* distance	1	<0.001	0.107	1.021	–
	A* energy	1	<0.001	1.083	0.648	–
	A* risk	1	<0.001	0.011	0.887	–
	MOPSO	12	1.505	1.2061	0.017	1.027
S3 (500)	NAMOA* (Tier 1)	115	33.761	1.1235	0.000	0.666
	A* distance	1	0.001	0.062	0.861	–
	A* energy	1	0.002	0.674	0.442	–
	A* risk	1	0.002	0.013	1.116	–
	MOPSO	38	4.326	1.1146	0.043	0.814

NAMOA* has an IGD of zero in every scenario, so its front contains every point of the reference front. That is what an exact method should give, and it confirms that the pruning rules do not throw away non-dominated routes. MOPSO recovers 12 of the 16 routes on S2 and 38 of the 115 on S3, and its HV is close to that of NAMOA*, with a gap of 0.0004 on S2 and 0.009 on S3.

The HV gap is small because MOPSO's archive still reaches the extremes of the front while missing most interior points, and interior points are the ones Tier 2 needs for mixed payloads. For this project the IGD and front-size columns say more than HV. NAMOA* also spreads its points more evenly, with Δ of 0.666 against 0.814 on S3. On S1 both methods find the same two-route front, which tells us the smallest scenario is too easy to separate them.

The single-objective A* routes reach a high HV only when their one objective happens to dominate the volume, as A* energy does on S2. None of them can

cover the front, and their IGD stays between 0.44 and 1.12. Figure 4.1 plots HV per method.

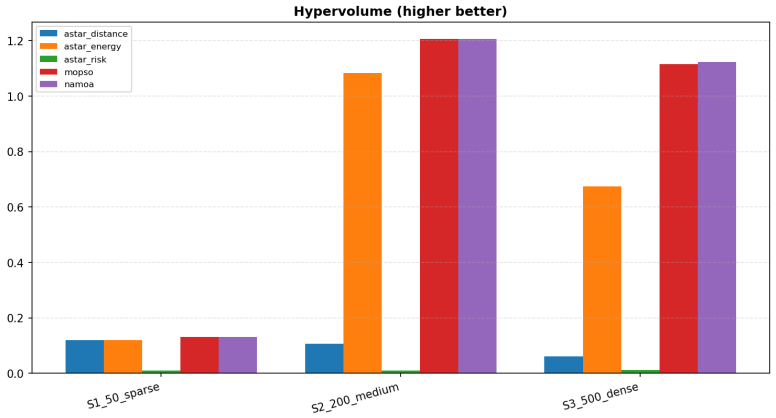


FIGURE 4.1: Hypervolume per method and scenario (first pass)

Exactness has a price. On S3, NAMOA* took 33.8 s against 4.3 s for MOPSO, , and the runtime grows quickly with front size. Tier 1 runs once per region and weather window, not once per mission, so half a minute is acceptable, but it is still the main performance target for the next pass. Options include front thinning, a tighter heuristic and a coarser lattice over open areas.

On S2 the HV values of 1.2065 and 1.2061 are too close to claim superiority without the 30 seeded MOPSO repetitions and a statistical test, which the next pass will run. The IGD result does not have this problem, since an IGD of zero cannot be beaten.

4.4.2 Effect of wind speed and direction

Table 4.5 shows the result of experiment E2.

The first observation is that direction alone matters a great deal. At the same speed, the cheapest route on the front needs 24.58 energy units with the wind from the

TABLE 4.5: Effect of wind direction on the S2 front (25 km/h wind, 35 km/h gust)

Wind from	Front	Labels	Min ε	Min ρ	Fragile pick (d, ε, ρ)	Distinct
Calm	36	869	24.41	0.07	(26.78, 26.92, 0.11)	4
0° (N)	14	1019	31.01	3.24	(26.16, 32.82, 3.40)	2
45° (NE)	6	545	31.34	1.89	(26.16, 33.63, 1.89)	2
90° (E)	11	876	30.94	3.31	(26.16, 32.75, 3.45)	2
135° (SE)	14	1100	27.13	4.27	(26.16, 29.59, 4.42)	2
180° (S)	14	989	26.18	3.24	(26.16, 27.99, 3.40)	2
225° (SW)	6	482	24.58	1.89	(26.16, 26.87, 1.89)	2
270° (W)	11	801	26.22	3.31	(26.16, 28.03, 3.45)	2
315° (NW)	14	1084	27.21	4.27	(26.16, 29.67, 4.42)	2

south-west and 31.34 with it from the north-east, a difference of 27.5%. A planner that used wind speed without direction would assign both cases the same cost.

Risk and energy also respond to different properties of the wind. Opposite directions (0° and 180°, or 45° and 225°) give the same minimum risk, because crosswind depends only on the axis of the wind. They give very different minimum energy, because the headwind penalty depends on which way along that axis the drone is flying. This is what Equation (3.5) predicts, and it is why the model needs both components.

Wind also changes the shape of the front. The front shrinks from 36 routes in calm air to between 6 and 14, and the four presets land on only two distinct routes. Under a steady wind, edges with strong crosswind are both costlier and riskier, so energy and risk become correlated and fewer real trade-offs are left. The fragile medical pick is the lowest-risk option on every windy front, but its energy varies from 26.87 to 33.63 with direction. Figure 4.2 repeats the experiment in 15° steps. The minimum energy moves between 24.6 and 31.6 units, a 28.5% swing, and the front holds between 6 and 15 routes.

The collapse from four distinct picks to two may be the right answer for a front with fewer real trade-offs, or a side effect of min-max normalisation on a short

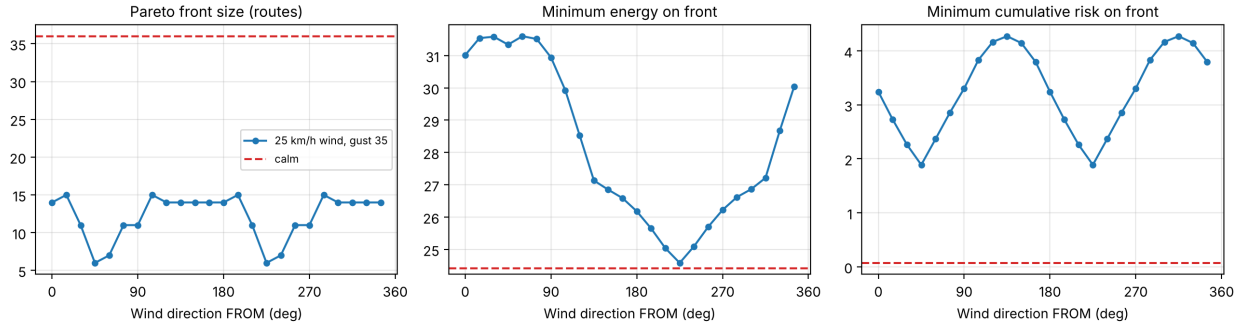


FIGURE 4.2: Front size, minimum energy and minimum risk on S2 as a 25 km/h wind turns through 360°; the dashed line is calm air

front. The next pass will test both, and the answer may change Tier 2.

4.4.3 Directional edge costs

Table 4.6 gives the result of E3.

TABLE 4.6: Directional edge costs on a flat lattice, 25 km/h wind from 90°

Edge heading	0°	45°	90°	135°	180°	225°	270°	315°
ε/d	1.104	1.270	1.278	1.270	1.104	1.074	1.000	1.074
ρ_{edge}	0.225	0.170	0.038	0.170	0.225	0.170	0.038	0.170

East-bound edges fly into the wind and cost 27.8% more energy per kilometre than west-bound edges, which fly with it and cost exactly their distance, since a tailwind earns no credit. North- and south-bound edges take the full crosswind and carry six times the risk score of east-west edges (0.225 against 0.038); the 0.038 left on the east-west edges comes only from the gust term. On this flat lattice the front still has a single route, because the straight diagonal is best on every objective. Trade-offs appear only when the wind interacts with terrain, buildings or restricted zones, which is what the ablation shows.

4.4.4 Contribution of each risk layer

Table 4.7 gives the result of E4. These runs used the fixture layers, so the numbers describe how each layer acts on the planner, not real-world values.

TABLE 4.7: Ablation on the 6 km lattice (fixture layers, 30 km/h wind from 225°)

Variant	Front	Time (s)	Min ε	Min ρ	Max ρ	Mean ρ_{edge}
All layers	196	11.35	8.82	8.85	13.12	0.199
No wind	205	5.38	7.84	0.10	4.24	0.061
No terrain	56	1.36	8.81	8.81	13.08	0.198
No buildings	190	8.32	8.82	8.85	12.26	0.192

In this setting wind is the largest single driver of risk. Removing it lowers the mean edge risk from 0.199 to 0.061 and the risk of the safest route from 8.85 to 0.10, and it lowers the minimum energy by 11%. Terrain mostly drives the size of the front. Without grades there are far fewer distinct trade-offs, 56 routes against 196, because altitude changes are what separate otherwise similar paths. Buildings raise the upper end of the risk range.

4.4.5 Tier 2 selection

Table 4.8 gives the Tier 2 picks of E5 on the S2 and S3 fronts. The ranks are the selected route's position on each objective within the front, where 1 is best.

TABLE 4.8: Tier 2 picks on the S2 (16 routes) and S3 (115 routes) fronts

Front	Preset	d	ε	ρ	Rk d	Rk ε	Rk ρ	Point type	ms
S2	Fragile medical	19.45	20.94	0.69	7	5	10	non-extreme	0.030
	Emergency medicine	19.15	20.56	0.81	3	1	14	min-energy	0.025
	Disaster relief	19.15	20.56	0.81	3	1	14	min-energy	0.022
	Mixed cargo	19.22	20.66	0.75	4	2	13	non-extreme	0.023
S3	Fragile medical	41.02	45.86	1.92	91	81	25	non-extreme	0.144
	Emergency medicine	37.11	44.71	6.56	22	26	94	non-extreme	0.140
	Disaster relief	37.52	44.27	5.63	28	1	88	min-energy	0.136
	Mixed cargo	39.90	45.06	2.74	84	59	32	non-extreme	0.136

On the 115-route S3 front, the four presets choose four different routes, and three of them are non-extreme points. Disaster relief takes the minimum-energy route, which matches its dominant weight.

The emergency medicine pick deserves a closer look. It is only the 22nd-shortest route, yet it is just 1.9% longer than the shortest route on the front. Its weights are dominated by distance, but not entirely, so the utility accepts a very small distance penalty for a lower risk. A single-objective planner cannot make that trade, and a fixed-weight planner would make the same trade for every payload.

The fragile medical and emergency picks show the size of the trade-off Tier 2 is choosing between. The fragile route is 10.5% longer than the emergency route (41.02 against 37.11) and carries 71% less cumulative risk (1.92 against 6.56). The fragile pick is 25th of 115 on risk rather than first, because its distance and energy weights are not zero, and the routes with even lower risk cost much more distance. Mixed cargo lands between the two on every objective, which is what its balanced weights ask for. Every selection took about 0.14 ms on S3, thousands of times inside the one-second target.

On the smaller S2 front the picks are less spread out. Emergency medicine and disaster relief share the minimum-energy route, which on this graph is also nearly the shortest. This is the same pattern as in the wind experiment, and it points to front size as the factor that limits how many distinct choices Tier 2 can make.

Figure 4.3 shows the four presets on a 2 km lattice (11-route front) with OpenStreetMap roads drawn underneath for reference. The fragile medical and mixed cargo presets take a slightly longer path (2.6 km) with a cumulative risk of

1.57, while emergency medicine and disaster relief take 2.5 km paths with risks of 1.88 and 1.89.

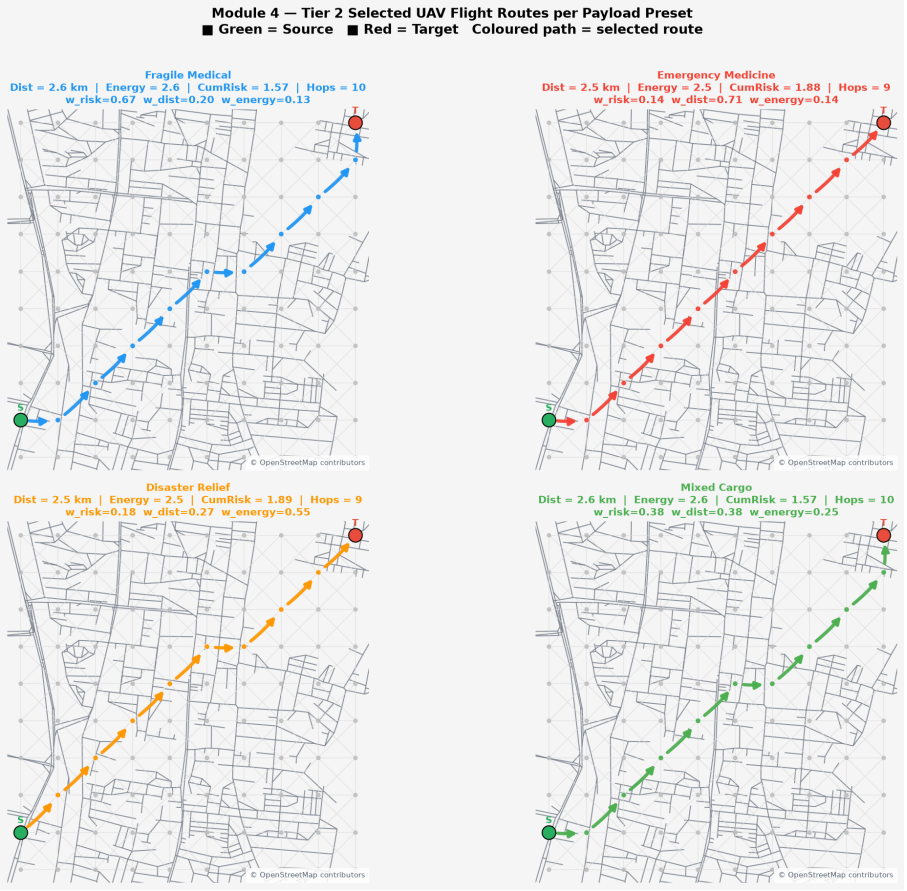


FIGURE 4.3: Tier 2 routes for the four presets on the 2 km lattice

The first monotonicity check reported 660 violations on the smallest benchmark, but it had compared selections across scenarios; grouped by scenario, the count is zero.

4.4.6 Simulation results

In E6 the SITL mission runner flew planner missions against ArduCopter SITL. Mission upload, read-back, the switch to AUTO, the landing and the disarm all succeeded. The cross-track root-mean-square error between the flown and planned

tracks was 0.11–0.13 m, and the landing offset from the destination pad was 0.01–0.02 m. These numbers show that the mission chain from Tier 2 to the autopilot is correct, including the altitude frame, which is easy to get wrong over uneven terrain.

Figure 4.4 shows a Tier 2 mission flown in AUTO inside the Gazebo city world, with the MAVProxy console on the left and the mission in QGroundControl on the right. The quantitative results above come from the runs with ArduPilot's built-in physics. The Gazebo world has no wind yet, and the battery constants are still placeholders, so logged comparisons across wind cases in Gazebo are planned for the next pass.

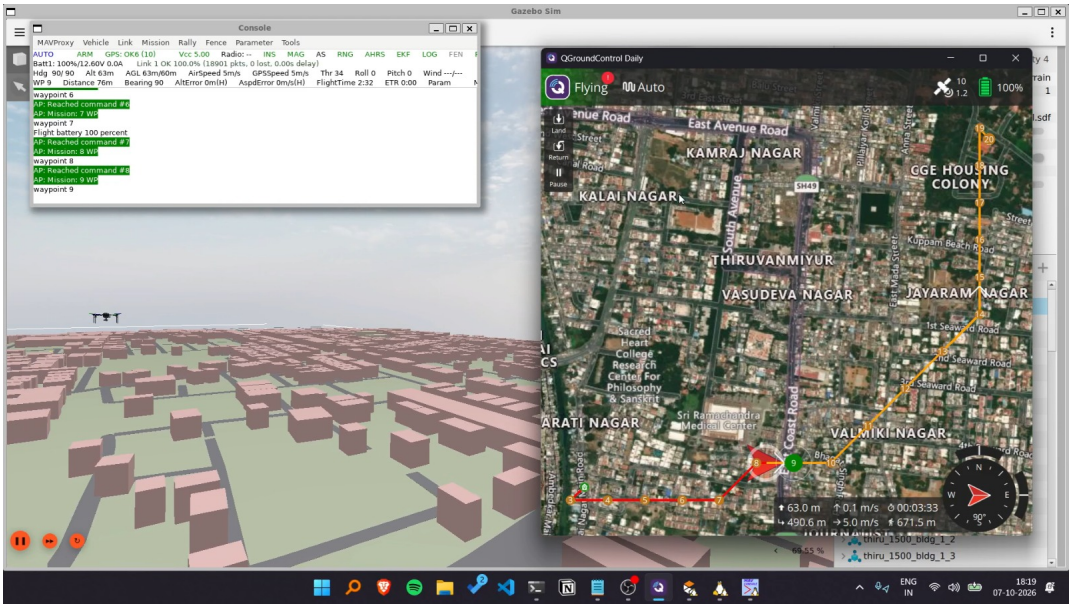


FIGURE 4.4: Gazebo Harmonic and ArduPilot SITL flying an uploaded Tier 2 mission in AUTO: MAVProxy console (left), QGroundControl mission view (right)

4.4.7 Comparison with existing work

Table 4.9 compares the system with the planners reviewed in Chapter 2.

TABLE 4.9: Comparison with the reviewed planners

Property	**Chokkalingam and Ganesan [4]**	**Ren et al. [18]**	**Deng et al. [7]**	**Tsoutsouras et al. [21]**	**This work**
Front construction	POPSO, stochastic	CDNSGA-II, stochastic	MS-INSGA2, stochastic	None; one trajectory	NAMOA*, exact and deterministic
Objectives	Length, energy, safety, obstacle penalty, weather factor	Distance, safety	Length, collision risk, smoothness	Deadline misses under energy and mass limits	Distance, energy, risk (11 components)
Final route selection	Weighted sum, weights set per mission	Crowding distance (k nearest)	Not discussed	Not applicable	Utility from payload tags, argmax over the full front
Payload in selection	No	No	No	Mass in the power model only	Fragility, weight class, urgency, priority
Wind	Speed-only factor, uniform	Not modelled	Future work	Not modelled	Headwind and crosswind per edge, gust, shear, weather gate
Front reuse	Re-run per mission	Re-run per mission	Re-run per task	Not applicable	Stored front with drift-based refresh

The comparison is qualitative for now, because their datasets and code are not public. The next pass applies their selection rules to our fronts and scores them with the payload-alignment measure of Section 3.6.

CHAPTER 5

SOCIAL IMPACT AND SUSTAINABILITY

5.1 SOCIAL IMPACT

The people who gain most from drone delivery are those far from a hospital or cut off by an emergency: patients in rural clinics waiting for blood or antivenom, towns isolated by a flood, and hospitals that send samples to a central laboratory every day. Zipline's network in Rwanda [9] and the hospital sample flights in Switzerland [2] show that such services already exist. What our work adds is a way for each flight to suit its cargo. Urgent antivenom can take the shortest corridor and a fragile sample a calmer one, without the operator knowing how the planner works.

This matters most where trained staff are scarce. The person launching a drone at a district hospital or a relief camp is likely to be a nurse, a volunteer or a logistics worker, not a route-planning expert. Our system asks four simple questions about the payload (how fragile, how heavy, how urgent, how important) and handles the rest.

A delivery drone also flies over people who never agreed to it. Population exposure, restricted zones, water and building proximity are part of every edge's cost, nodes above the legal ceiling are removed, and a weather gate stops flights in storms, heavy rain, poor visibility or strong gusts. It keeps the planner from routing over a crowded market to save a few seconds.

Every Tier 2 decision is recorded with its tags, weights, route and any HOLD reason, so it can be reproduced later. The benefit depends on honest limits, though. The fragility tag is a precaution, not a proven protection [14, 19, 20], and until the risk weights are calibrated, the risk numbers should be read as rankings between routes, not as probabilities of harm. Outdoor flights in Phase II will follow the Drone Rules, 2021 [13].

5.2 SUSTAINABILITY

For short, light deliveries, a small electric drone replaces a road trip by van or motorcycle and produces no exhaust where it flies. The planner adds to this in two ways. Its energy objective sees wind direction, which alone changed the energy of the cheapest route by 27.5% in our study, so it can avoid strong headwinds. And the payload weight tag gives energy more weight when the load is heavy, which is when battery use matters most. The feasibility filter refuses routes that would eat into the 20% battery reserve, which protects both the mission and the battery's life.

The system is also light on computing. Tier 1 runs on an ordinary laptop CPU with no GPU or cloud service, once per region and weather window, and its front is reused by every mission until it expires. Tier 2 takes a fraction of a millisecond and can run on a Raspberry Pi on board.

Everything the system depends on is open data or open-source software, so there are no licence fees and the method can be set up for any region. With all constants

in one file and about 265 offline tests, it can be recalibrated as real flight data comes in.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project set out to let the payload decide the route. Existing multi-objective UAV planners build good Pareto fronts and then choose from them with a rule that knows nothing about the cargo, or with weights an expert re-tunes for every mission. We separated the two problems. Tier 1 builds an exact NAMOA* front over a three-dimensional, wind-aware airspace lattice once per region and weather window, and stores it with what is needed to tell when it has gone stale. Tier 2 turns four payload tags into objective weights with an explicit formula, scores every route on the front and returns the best one, or a HOLD when the weather or the battery rules the mission out. The chosen route is exported as an ArduPilot mission and has been flown in SITL and in a Gazebo city world.

The first results are encouraging, with clear caveats. The Tier 1 front was exact in every benchmark scenario, while MOPSO found 38 of 115 routes on the largest. On that front the four reference payloads received four different routes, three of them non-extreme, in about 0.14 ms each. Wind direction alone moved the energy of the cheapest route by 27.5% at a fixed speed, and in SITL the flown track stayed within 0.13 m RMS of the plan. On the other hand, under a steady wind the front shrank and the presets collapsed onto two routes, MOPSO ran at reduced settings, the ablation used fixture data, and the risk weights and battery constants are still placeholders.

6.2 FUTURE WORK

The rest of Phase I is a set of passes through the evaluation cycle. We will run the full benchmark (four graphs, nine wind cases, MOPSO at 100 particles, 200 iterations and 30 repetitions) with per-repetition statistics and a significance test; fix the density labels and the monotonicity grouping; study why the front collapses under steady wind, testing other normalisations and weight multipliers; repeat the ablation and Tier 2 studies on live map data; compare Tier 2 with crowding distance, knee point, shortest route and fixed weights over all 225 tag combinations; close the two dashboard defects; and reduce the Tier 1 runtime on large graphs. On the simulation side, wind will be added to the Gazebo world, the battery model will be calibrated per payload weight class, and each preset will be flown under several wind cases with all flight metrics logged.

Phase II moves the system onto a Pixhawk-equipped F450 quadcopter with a Raspberry Pi 5 companion computer and a servo payload release. Tier 2 will run on board, and outdoor flights will measure route adherence and the time from tag entry to mission start. Further out, the two-tier idea could extend to several drones sharing one front, to multi-drop routes and to in-flight replanning when the weather changes faster than the refresh window.

REFERENCES

1. ArduPilot Development Team. (2026). *ArduPilot software in the loop (SITL) simulator*. https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html

2. Avionics International. (2019, August 8). *Drone delivery crash in Switzerland raises safety concerns as UPS forms subsidiary*. https://www.aviationtoday.com/2019/08/08/drone-delivery-crash-in-switzerland-raises-safety-concerns/

3. Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, *65*, 126–139. https://doi.org/10.1016/j.compenvurbsys.2017.05.004

4. Chokkalingam, K., & Ganesan, R. (2026). Dynamic parameter adaptation in Pareto-optimal particle swarm optimization for efficient UAV path planning. *RAIRO – Operations Research*, *60*(1), 281–305. https://doi.org/10.1051/ro/2025159

5. Coello Coello, C. A., Toscano Pulido, G., & Salazar Lechuga, M. (2004). Handling multiple objectives with particle swarm optimization. *IEEE Transactions on Evolutionary Computation*, *8*(3), 256–279. https://doi.org/10.1109/TEVC.2004.826067

6. Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, *6*(2), 182–197. https://doi.org/10.1109/4235.996017

7. Deng, J., Zhang, H., Feng, D., Yu, Z., & Liu, Z. (2026). Multi-objective UAV trajectory planning algorithm based on the hybrid MS-INSGA2 strategy. *Journal of King Saud University – Computer and Information Sciences*, *38*, Article 562. https://doi.org/10.1007/s44443-026-00930-0

8. Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)* (pp. 11–15). https://doi.org/10.25080/TCWV9851

9. IEEE Spectrum. (2019). *In the air with Zipline's medical delivery drones*. https://spectrum.ieee.org/in-the-air-with-ziplines-medical-delivery-drones

10. Joint Authorities for Rulemaking of Unmanned Systems. (2019). *JARUS guidelines on specific operations risk assessment (SORA)* (JAR-DEL-WG6-D.04, Edition 2.0).

11. Mandow, L., & Pérez de la Cruz, J. L. (2010). Multiobjective A* search with consistent heuristics. *Journal of the ACM*, *57*(5), Article 27. https://doi.org/10.1145/1754399.1754400

12. Meier, L., et al. (2013). *MAVLink: Micro air vehicle communication protocol*. https://mavlink.io

13. Ministry of Civil Aviation, Government of India. (2021). *The Drone Rules, 2021* (G.S.R. 589(E)). Gazette of India.

14. Oakey, A., Waters, T., Zhu, W., Royall, P. G., Cherrett, T., Courtney, P., Majoe, D., & Jelev, N. (2021). Quantifying the effects of vibration on medicines in transit caused by fixed-wing and multi-copter drones. *Drones*, *5*(1), 22. https://doi.org/10.3390/drones5010022

15. Open-Meteo. (2026). *Free weather API and elevation API*. https://open-meteo.com

16. Open Robotics. (2023). *Gazebo Harmonic*. https://gazebosim.org

17. Peterson, E. W., & Hennessey, J. P. (1978). On the use of power laws for estimates of wind power potential. *Journal of Applied Meteorology*, *17*(3), 390–394. https://doi.org/10.1175/1520-0450(1978)017<0390:OTUOPL>2.0.CO;2

18. Ren, Q., Yao, Y., Yang, G., & Zhou, X. (2019). Multi-objective path planning for UAV in the urban environment based on CDNSGA-II. In *Proceedings of the IEEE International Conference on Service-Oriented System Engineering (SOSE)*. https://doi.org/10.1109/SOSE.2019.00059

19. Stierlin, N., Loertscher, F., Renz, H., Risch, L., & Risch, M. (2024). Preanalytic integrity of blood samples in uncrewed aerial vehicle (UAV) medical transport: A comparative study. *Drones*, *8*(9), 517. https://doi.org/10.3390/drones8090517

20. Theobald, K., Zhu, W., Waters, T., Cherrett, T., Oakey, A., & Royall, P. G. (2023). Stability of medicines transported by cargo drones: Investigating the effects of vibration from multi-stage flight. *Drones*, *7*(11), 658. https://doi.org/10.3390/drones7110658

21. Tsoutsouras, V., Story, J., & Stanley-Marbell, P. (2020). *Payload-mass-aware trajectory planning on multi-user autonomous unmanned aerial vehicles* (arXiv:2001.02531). arXiv. https://doi.org/10.48550/arXiv.2001.02531

22. Zitzler, E., & Thiele, L. (1999). Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Transactions on Evolutionary Computation*, *3*(4), 257–271. https://doi.org/10.1109/4235.797969

APPENDIX A

CONFIGURATION PARAMETERS

All tunable constants live in `config.py`. Table A.1 lists the main parameters of the airspace graph and cost model, and Table A.2 those of the front store and Tier 2. Values marked provisional are placeholders that will be replaced by calibrated values.

TABLE A.1: Main airspace graph and cost model parameters

Parameter	**Value**	**Meaning**
`BOX_M`, `LATTICE_SPACING_M`	6000 m, 200 m	Lattice extent and spacing
`H_CRUISE_AGL`, `CLEARANCE`	60 m, 15 m	Cruise height, margin above roofs
`H_MAX_AGL`	120 m	Legal ceiling above ground
`V_CRUISE_KMH`	36 km/h	Airspeed used in the wind terms
`BETA_HEAD`, `BETA_CROSS`	0.40, 0.15	Headwind and crosswind energy coefficients
`V_MAX_SAFE_KMH`, `K_GUST`	40 km/h, 0.5	Wind risk saturation speed, gust weight
`ALPHA_SHEAR`	0.25	Power-law wind shear exponent
`EDGE_SAMPLE_STEP_M`	25 m	Sampling step along each edge
`NFZ_SOFT_LAMBDA_M`	150 m	Decay length of the no-fly buffer
`NO_GO_GUST_KMH`	45 km/h	Gust limit of the weather gate (provisional)
`RISK_WEIGHTS`	Table 3.1	Weights of the eleven risk components (uncalibrated)

TABLE A.2: Front store and Tier 2 parameters

Parameter	**Value**	**Meaning**
Front validity window	3 h	Front expires after this time
Wind speed drift	10 km/h	Rebuild when wind speed moves more than this
Wind direction drift	45° (wind > 10 km/h)	Rebuild when wind turns more than this
Gust drift	15 km/h	Rebuild when gust moves more than this
Tier 1 expansion cap	200,000 (S1/S2/S3: 80k, 180k, 260k)	Guards against runaway search
Payload masses m_W	0.25, 0.5, 1.0 kg	Nominal mass of weight classes 1, 2, 3
m_{uav}, battery, reserve	1.5 kg, 74 Wh, 20%	Airframe mass and battery (provisional)
k_{cal}	6 Wh per unit	Energy proxy to watt-hours (provisional)
Priority threshold	$P_{\text{lvl}} \geq 4$	Dominant weight amplified by 10%

APPENDIX B

AUTOMATED TEST SUITES

Both code bases have offline unit-test suites that need no network and no API key. The planner suite has about 220 tests and the simulation suite 45. Table B.1 lists what they check.

TABLE B.1: Test suites and what they verify

Test file	**What it checks**
`test_wind.py`	Headwind sign (wind from 90° is a headwind for an east-bound edge), crosswind, power-law shear, hourly row selection
`test_lattice.py`	Lattice size, 8-neighbour edges, east-bound edges cost more than west-bound edges under an east wind
`test_terrain.py`	DEM provider chain and cache, node altitude rule, every node within 120 m above ground
`test_risk.py`	Risk components in $[0, 1]$, restricted-zone node and edge removal, soft buffer, weather gate
`test_tier1_front.py`	NAMOA* front equals a brute-force front over all simple paths on a small graph; no route dominates another; every path valid
`test_store.py`	Reuse, expiry, graph-hash change, each drift threshold, identity check
`test_tier2_payload.py`, `test_tier2_hardening.py`	Weight formula, amplification, non-extreme pick for mixed cargo on a stored lattice front, monotonicity, validation errors, latency under 1 s, battery HOLD

Test file	**What it checks**
`test_evaluation_metrics.py`	HV, IGD and spread on hand-computed cases; alignment score; competing pickers
`test_mission_export.py`	QGC WPL 110 round trip, collinear merge, frame 3 altitude relative to home
`test_ablation.py`, `test_overpass.py`, `test_dotenv.py`, `test_dashboard_helpers.py`	Ablation smoke run, Overpass retry and parsing, key loading, dashboard helpers
`Simulation/tests/*`	Bundle loading and frames, clearance checker, mission export (take-off, route, land), world builder geometry

APPENDIX C

CODE ORGANISATION

The planner is written in Python as flat scripts, one file per module, plus a configuration file, an environment-data file and entry-point scripts. The simulation lives in a separate folder and imports the planner modules through the Python path. Every tunable constant (region, flight heights, wind coefficients, risk weights, battery and mass values, drift thresholds, Overpass endpoints) lives in one file, `config.py`, so an experiment changes a value in one place. API keys are read from a `.env` file by the entry-point scripts only, so the test suites never need a key or a network connection. Table C.1 maps the source files to the modules.

TABLE C.1: Source files and the modules they implement

File	Module	Role
`config.py`	all	Every tunable constant
`env_data.py`	1	Weather, DEM providers, OSM layers, spatial indexes, key loader
`overpass_client.py`	1	Multi-endpoint Overpass fetch with retry
`module1_graph_builder.py`	1	Synthetic and lattice builders, cost pipeline
`module2_tier1_moastar.py`	2	NAMOA* engine, dominance helpers, thinning
`pareto_store.py`	2	Front store, graph hash, drift, region runs
`module3_baselines.py`	3	A* variants, weighted-sum A*, MOPSO
`module4_tier2_payload.py`	4	Weight bridge, selector, feasibility, export
`module5_evaluation.py`	5	HV, IGD, spread, Tier 2 evidence, sweep
`run_benchmarks.py`, `run_ablation.py`	5	Benchmark and ablation runners
`mission_export.py`, `dashboard.py`	6	QGC WPL 110 export, operator interface
`run_demo_syn.py`, `run_demo_osm.py`	1–4	End-to-end demonstrations with figures
`Simulation/*.py`	7	Bundle, clearance, world, mission, runner, logger, evaluator